



廈門大學
XIAMEN UNIVERSITY

本科期末报告

(主修专业)

Cocoon Breaker

茧房终结者

姓名: 廖梓强 22920212204414

曹丹颖 22920212204347

邓明宇 22920212204365

薛婉如 22920212205270

学院: 信息学院

专业: 数字媒体技术

年级: 2021级

课程名: 人工智能导论

任课教师: 姚俊峰 (教授)

二〇二四年六月十六日

Abstract

In today’s digital age, personalized content curation and recommendation algorithms have led to a common problem known as the “Information Cocoon”. This phenomenon occurs when individuals become increasingly isolated within their own information bubbles. To address this issue, we have developed a project called “Cocoon Breaker” that utilizes advanced AI technologies. The goal of our project is to help users break out of their information cocoons by gathering and organizing diverse information. By doing so, we aim to provide users with a more comprehensive understanding of various topics and social phenomena. Our system consists of several key components, including text embedding, sentiment analysis, video processing, and fact verification. These components are integrated into a user-friendly platform.

Our system offers detailed video summaries, event analysis, and the presentation of diverse viewpoints. These features aim to enhance users’ information literacy and promote social cohesion. We have designed the system with a modular architecture, allowing for easy integration and asynchronous processing. We have also developed effective deployment strategies to ensure smooth operation. Through extensive evaluation, we have demonstrated the system’s effectiveness in improving user access to diverse perspectives. We believe that our project has the potential to foster an informed and harmonious society. In the future, we plan to optimize the analysis workflow, expand support for multiple platforms, and incorporate interactive features to further enhance user experience and engagement.

Keywords: Information Cocoons, Artificial Intelligence, Large Language Model, Deep Learning

Contents

1	Introduction	1
1.1	Background	1
1.1.1	Concept of Information Cocoons	1
1.1.2	Social Context of Information Cocoons	2
1.1.3	Dangers of Information Cocoons	2
1.1.4	Conclusion	3
1.2	User Survey	3
1.3	Analysis of Similar Products	5
1.4	Project Description	8
1.4.1	Project Goals	8
1.4.2	Main Features	9
1.4.3	Project Module Design	10
2	Related Works	12
2.1	Text Embedding	12
2.1.1	BERT: A Revolutionary Approach	12
2.1.2	BERT Variants and Multilingual Capabilities	12
2.1.3	Applications and Benefits	13
2.1.4	Clustering Method	14
2.2	LLMs	15
2.2.1	Two Major Tasks	15
2.2.2	Traditional Methods	16

2.2.3	Large Model Approaches	17
2.2.4	Principles of Large Models	20
2.2.5	Common Large Models Comparison	21
2.2.6	Prompt Engineering	22
2.3	Kity Minder	24
2.4	Sentiment Analysis	25
2.4.1	Introduction to Sentiment Analysis	25
2.4.2	Multilingual Sentiment Analysis Model	26
3	Architecture Design	27
3.1	Overall Architecture	29
3.2	Web Plugin	31
3.3	Frontend and Backend Design	32
3.3.1	Frontend Design	32
3.3.2	Backend Design	36
3.4	Video Processing	39
3.4.1	Video Information Retrieval	39
3.4.2	Video Text Extraction	42
3.4.3	Video Comment and Danmu Crawling	45
3.5	Online Search	48
3.6	GPT Analysis	50
3.6.1	Video Summary	50
3.6.2	Extracting Search Keywords	52
3.6.3	Event Analysis	52
3.6.4	Fact Verification	54
3.7	Mind Map Generation	56
3.8	Opinion Analysis	59
3.8.1	Sentiment Analysis	59
3.8.2	Clustering Analysis	61

4	Evaluation	63
4.1	Stability Experiment	63
4.2	Effectiveness Evaluation	65
4.3	Duration Evaluation	66
5	Future Outlook	68
5.1	Model Cost Issue	68
5.2	User Account	69
5.3	Optimization of Analysis Workflow	70
5.4	Multi-Platform Support	71
5.5	Hot Topic Recommendations	73
5.6	Interactive System	74
6	Conclusion	77

Chapter 1: Introduction

1.1 Background

In the current digital era, the prevalence of personalized content curation and recommendation algorithms has led to the emergence of the “information cocoon” phenomenon. This concept, first introduced by Harvard Professor Cass Sunstein, refers to the situation where individuals are increasingly isolated within their own informational niches due to selective exposure to content that aligns with their pre-existing beliefs and preferences[1]. As the internet has evolved, especially with the advent of platforms like TikTok, Toutiao, and Zhihu, the issue of information cocoons has become more pronounced. These platforms utilize sophisticated algorithms to create detailed user profiles and deliver highly personalized content, which, over time, confines users to a narrow circle of information[2].

1.1.1 Concept of Information Cocoons

The term “information cocoon” was coined by Sunstein to describe the process by which individuals, through selective exposure to information sources that align with their interests, gradually become encased in a metaphorical cocoon of homogeneous information[1][2]. This process is facilitated by the use of recommendation algorithms that track user behavior and preferences, subsequently curating content that is likely to keep the user engaged. Over time, this leads to a narrowing of the information landscape, where users are repeatedly exposed to similar viewpoints and content, effec-

tively isolating them from contrasting information[3].

1.1.2 Social Context of Information Cocoons

In today’s interconnected world, the internet serves as a vast and complex information dissemination platform. It functions not only as a tool for personal information processing but also as a medium for interpersonal, group, and organizational communication[3]. However, the rise of social media and personalized content platforms has significantly changed how information is consumed. The shift towards algorithm-driven content delivery has led to a scenario where users are bombarded with homogeneous information, reinforcing their existing views and limiting exposure to diverse perspectives[4]. This phenomenon is often referred to as the “information cocoon”, where individuals are trapped in a self-created bubble of information that reflects their personal biases and preferences[1].

1.1.3 Dangers of Information Cocoons

Information cocoons pose several significant risks to both individuals and society at large. Firstly, they exacerbate the phenomenon of group polarization, where users immersed in homogeneous information environments become more extreme in their views[4]. This is particularly evident in online communities, social networks, and discussion forums, where like-minded individuals reinforce each other’s opinions, leading to more radicalized group behavior[4][5]. Secondly, the prevalence of information cocoons undermines social cohesion by reducing the instances of shared experiences and common knowledge, which are essential for societal bonds[3]. As people increasingly rely on personalized content, their interactions with diverse viewpoints diminish, leading to a fragmented society with diminished social stickiness[3][5].

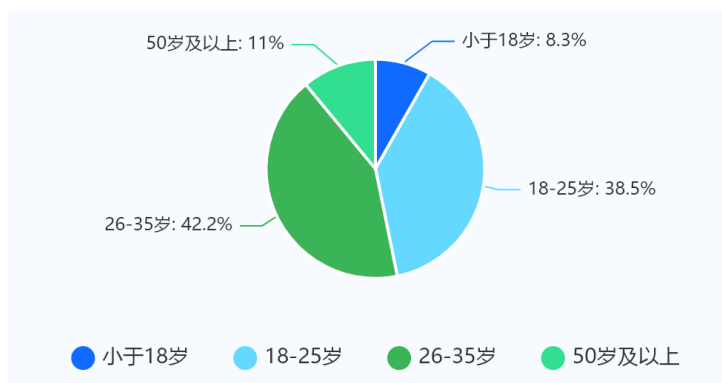
1.1.4 Conclusion

The information cocoon phenomenon represents a significant challenge in the digital age, requiring a multifaceted approach to address its implications. By understanding the mechanisms behind information cocoons and their impact on society, strategies can be developed to mitigate their negative effects. These strategies include enhancing algorithmic transparency, promoting media literacy, and encouraging exposure to diverse viewpoints. Addressing the issue of information cocoons is crucial for fostering an informed and cohesive society.

Therefore, with a sense of social responsibility, we have decided to use technological means to assist users in breaking out of their comfort zones at a lower cost, gaining access to more information beyond their information cocoons. We will utilize AI technology to aggregate and organize this information, helping users form a more comprehensive understanding of various matters and social phenomena. We believe that our system will play a positive role in promoting social harmony and stability.

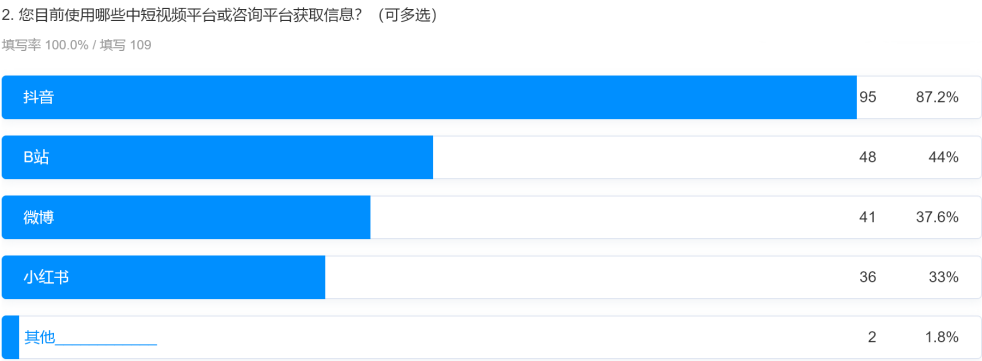
1.2 User Survey

We conducted a survey of 110 users, with the following distribution of ages:

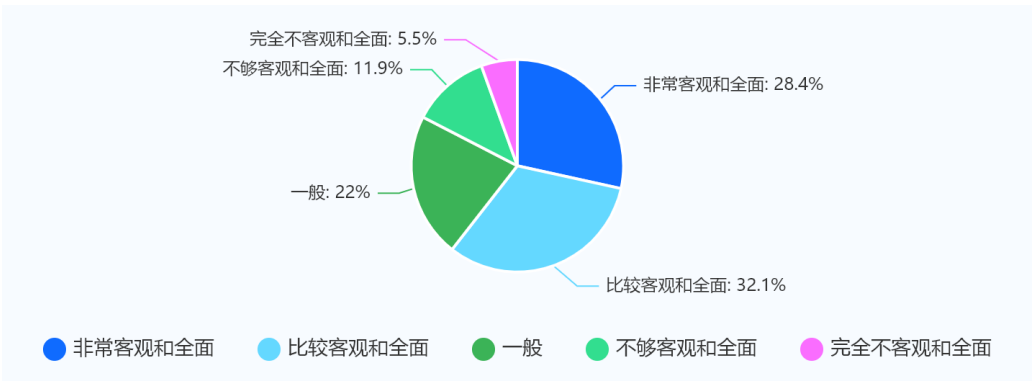


The survey results indicate that the primary sources of information for

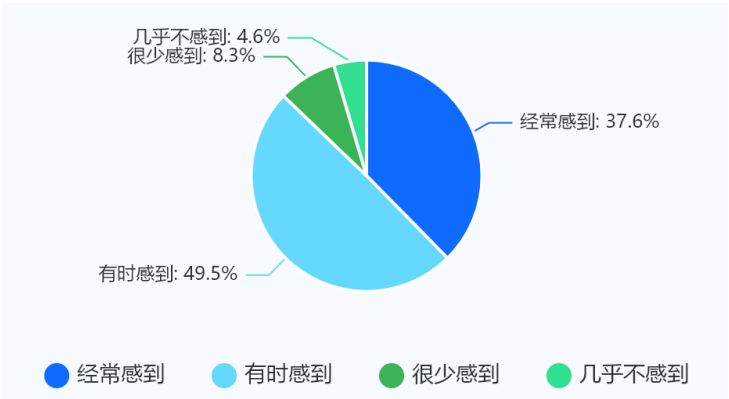
most Chinese users are TikTok, Bilibili, Weibo, and Xiaohongshu.



Although most users believe that modern information is sufficiently comprehensive, there are still some users who perceive that certain information may contain subjective views from the publishers, leading to incorrect dissemination of information.

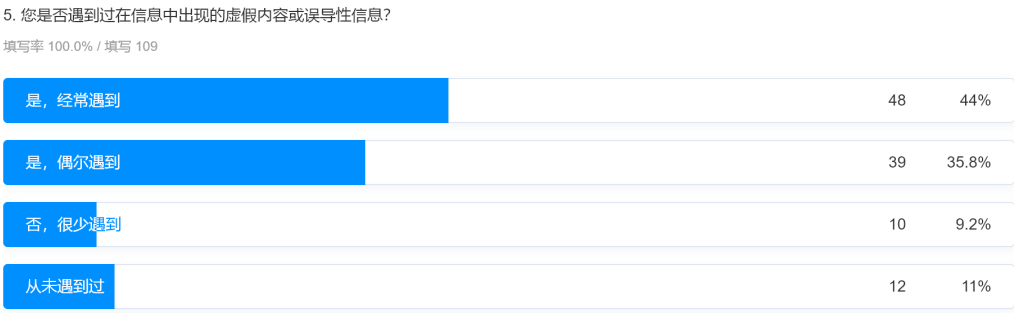


From the survey results, it is evident that the majority of users often feel overwhelmed by information overload when browsing the internet.

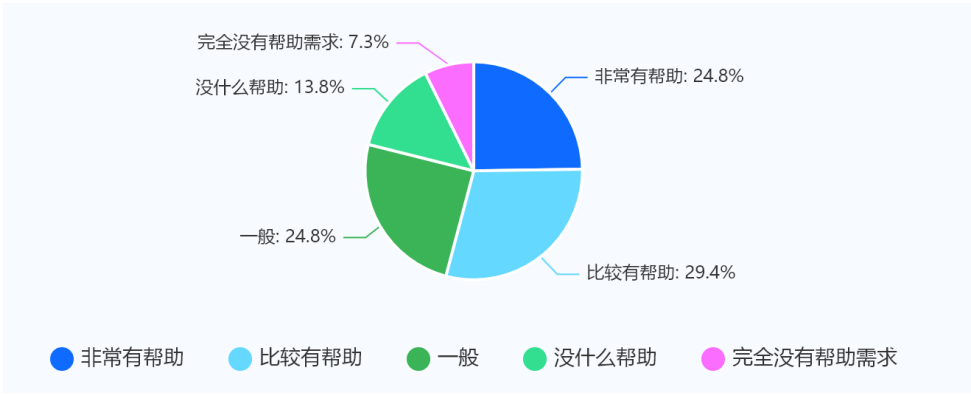


The research findings indicate that the majority of users have encoun-

tered false or misleading information in the content they consume.



Overall, the survey results indicate that the “Cocoon Breaker” project is helpful to a certain extent for the majority of users.



1.3 Analysis of Similar Products

In the current market, several platforms and products aim to address the issue of information cocoons and provide diverse content perspectives. Below are the major competitors and an analysis of their functionalities:

- **Braver:** Braver is a browser extension that displays multiple news viewpoints to help users break out of information cocoons. Its main advantages include being simple and easy to use, as it operates directly within the browser, and offering multiple perspectives for comparison. However, its functionality is limited, primarily focusing on news and lacking in-depth analysis and customization options.

- **Ground News:** Ground News provides multi-source news comparisons, showcasing how different media outlets report on the same event. The platform’s strengths lie in its ability to offer multi-source news comparisons, helping users understand different viewpoints, and its user-friendly interface. On the downside, Ground News primarily focuses on news, with less coverage of other types of information, and lacks interactive features.
- **Newsela:** Newsela is an educational platform that provides multi-source news content tailored to different reading levels. It is particularly beneficial for educational environments, helping students improve reading skills and media literacy. However, Newsela mainly targets the education market, making it of limited use for general users and not ideal for everyday information retrieval.
- **AllSides:** AllSides classifies news by political bias, presenting multiple perspectives on the same issue. This helps users understand different political standpoints and offers clear categorization. Nonetheless, AllSides focuses mainly on political news, with limited coverage of other information types, and has limited interactive features.
- **FlipFeed:** FlipFeed is a social media tool that allows users to view feeds from people with opposing viewpoints. It increases exposure to different perspectives and promotes strong social interaction. However, it relies on social media platforms, offering limited coverage, and lacks in-depth analytical functions.

Compared to the above competitors, our project offers unique advantages in the following areas:

- **Breaking Information Cocoons:** Our project, “Cocoon Breaker”, is not limited to news alone. It encompasses various types of information,

including social media, news, and videos. By leveraging AI technology, we aggregate information from multiple platforms to provide diverse content perspectives. This encourages users to engage with different viewpoints and break out of their information cocoons.

- **Identifying Information Authenticity:** We introduce an information authenticity verification feature that assists users in discerning the truthfulness of information. Utilizing AI algorithms and fact-checking databases, our system automatically detects and flags potential misinformation, helping users navigate through the vast amount of information available more critically.
- **Information Organization and Mind Map Generation:** Our project offers advanced features for information organization and mind map generation. By employing AI technology, we organize related information and generate intuitive mind maps. This helps users manage and understand complex information more effectively, facilitating a deeper comprehension and application of the knowledge they acquire.

Our project, “Cocoon Breaker”, not only aids users in breaking out of information cocoons but also includes features for identifying information authenticity and organizing information. These components make our project a comprehensive tool for enhancing information literacy and promoting a more informed and cohesive society. By harnessing advanced technologies and innovative approaches, we strive to provide a diverse, transparent, and secure information environment for users.

1.4 Project Description

1.4.1 Project Goals

Break Information Cocoons: This project aims to use technological means to help users access more diverse information beyond their usual sources and perspectives. Most people currently face a paradox: on one hand, they receive a large amount of current news and thus require more comprehensive information to aid their judgment; on the other hand, most of them do not possess professional investigative skills and have limited time to spend on fact-checking. Therefore, an unbiased, automated program can provide the necessary service in this regard.

Improve Information Literacy: Information literacy is an essential skill for modern citizens. It not only includes the ability to identify and obtain useful information but also encompasses evaluating the authenticity and reliability of information. Enhancing information literacy can help the public better cope with false information and misleading media reports, enabling them to make informed decisions. It also reduces the likelihood of the public being swayed or deceived. Overall, improving information literacy will significantly enhance the public's ability to navigate a complex information environment.

Promote Social Harmony: Social harmony relies on effective communication and mutual understanding between different groups. By promoting cross-cultural exchange and diversity education, misunderstandings and prejudices can be reduced, fostering coexistence and cooperation. For example, in diverse communities, organizing cultural exchange activities and dialogue platforms can help individuals from different backgrounds share experiences and perspectives, building mutual trust. Additionally, media and educational institutions can play a crucial role by spreading values of inclusion

and respect, reducing social division and antagonism. In summary, promoting social harmony not only contributes to social stability but also enhances overall happiness and social cohesion.

1.4.2 Main Features

Video Summarization: Video content is rich and intuitive, but its linear nature often makes it difficult to quickly search and review. Converting video content into text can help users quickly browse and understand the main information of the video, making it easier for them to refer back to and think critically about the content as needed. This textual summary can include the main points, key data, and important conclusions of the video, presented in a clear and concise manner, allowing users to access the necessary information without repeatedly watching the video. Additionally, text summaries can utilize keyword search functions to help users quickly find relevant content within a large number of videos, improving information utilization efficiency. This approach is particularly useful for fields such as education and research, which require repeated in-depth analysis and verification.

Fact Verification and Event Analysis: In an era of information overload, the accuracy of data, events, and scientific conclusions mentioned in videos is crucial. By verifying the facts mentioned in videos, users can be ensured they are not misled. This process includes verifying statistical data cited in the video, conducting timeline analyses of mentioned historical events, and fact-checking scientific conclusions. Furthermore, the program will search the internet for related information, providing a comprehensive analysis report that outlines the complete development of events. This not only helps users understand the authenticity of the video content but also offers more comprehensive background information, enhancing users' judgment

and information literacy.

Presentation of Diverse Opinions: In the process of information dissemination and communication, presenting diverse perspectives can effectively promote understanding and interaction between different opinion groups. By collecting and analyzing different viewpoints from multiple commonly used platforms, users can be provided with a multi-faceted understanding of the topic, preventing them from being confined to a single perspective. The presentation of diverse perspectives helps reduce the information bubble effect, promoting social inclusion and consensus building. Additionally, this approach can strengthen communication and interaction between different opinion groups, reducing misunderstandings and antagonism, and fostering social harmony.

1.4.3 Project Module Design

Module	Description
GPT Event Analysis	Obtain a query question, call Google API to search for information, crawl search results, and return textual event analysis and mindmap code.
Video Information Retrieval	Get Bilibili's BV number as input, automatically download video audio and convert it to text, while also retrieving description, title, tags, etc.
Video Content Summary	Summarize video content, mark important content timestamps, and assist users in extracting and organizing video information.
Search Keyword Summary	After converting video to text, verify factual information, organize events, and search for keywords.
Social Media Comment Crawling + Analysis	Retrieve related posts and comments from major social platforms and analyze them, returning the analysis results as HTML elements.
Frontend Design	Use HTML, CSS, JavaScript, etc., to build a beautiful and interactive frontend page.
Django Backend Deployment	Combine all modules into a complete project and deploy it, responding to HTTP requests from the frontend and returning new HTML file addresses.

Table 1.1: Project Module Design

Chapter 2: Related Works

2.1 Text Embedding

Word embedding is essential in NLP, converting text into numerical vectors for machine learning models. Traditional methods like one-hot encoding and word2vec advanced the field by providing dense word vectors, capturing semantic relationships to some extent. However, these static embeddings struggle with polysemy and contextual variations [6].

2.1.1 BERT: A Revolutionary Approach

Bidirectional Encoder Representations from Transformers (BERT) revolutionized word embeddings by generating dynamic embeddings, considering context from both directions [7]. This allows BERT to capture nuanced word meanings in various contexts, improving NLP task performance.

BERT's transformer architecture uses self-attention mechanisms for parallel processing and deep bidirectional understanding. Pre-trained on large corpora with masked language modeling (MLM) and next sentence prediction (NSP), BERT learns bidirectional context and sentence relationships [7].

2.1.2 BERT Variants and Multilingual Capabilities

A notable BERT variant is the bert-base-multilingual-cased model, extending BERT's capabilities to multiple languages. Pre-trained on Wikipedia pages in 104 languages, it effectively processes multilingual text.

The bert-base-multilingual-cased shares the original BERT-base architecture with 12 layers, 768 hidden units, and 12 attention heads, totaling 110 million parameters. Its training data includes text from 104 languages, and it preserves case information, aiding languages with case distinctions.

Here is the comparison of BERT and its multilingual variant highlights their differences and advantages:

Aspect	BERT (Original)	bert-base-multilingual-cased
Architecture	12 layers, 768 hidden units, 12 attention heads	12 layers, 768 hidden units, 12 attention heads
Pre-training Data	English Wikipedia, BooksCorpus	Wikipedia in 104 languages
Case Sensitivity	Uncased	Cased
Number of Parameters	110 million	110 million
Contextual Embeddings	Yes	Yes
Multilingual Support	No	Yes

Table 2.1: Comparison between BERT (Original) and bert-base-multilingual-cased

2.1.3 Applications and Benefits

The bert-base-multilingual-cased model excels in multilingual NLP tasks, offering consistent performance across languages. It benefits applications like machine translation, cross-lingual information retrieval, and multilingual sentiment analysis. BERT’s dynamic embeddings capture contextual word meanings, enhancing tasks requiring deep sentence semantics understanding.

The bert-base-multilingual-cased model advances NLP by combining BERT’s dynamic embeddings with multilingual processing. It is ideal for applications needing robust multilingual understanding, providing a valuable tool for researchers and practitioners.

2.1.4 Clustering Method

After completing the embedding vectors for the text, the next task is to classify these vectors effectively. To achieve this goal, we can employ various established methods, among which K-means clustering is widely used and popular.

K-means clustering is a well-known unsupervised learning algorithm. Its core idea is to partition the data points into K clusters, such that data points within each cluster are as similar as possible, while data points in different clusters are as dissimilar as possible. The algorithm iteratively optimizes the cluster center positions and the cluster assignments of data points to minimize the within-cluster variance.

In the clustering process, the choice of distance metric is crucial, as different metrics can significantly impact the clustering results. Common distance metrics include Euclidean distance and cosine distance.

Euclidean Distance

Euclidean distance measures the straight-line distance between two points and is suitable for cases where vector dimensions and measurement standards are consistent. The calculation formula is as follows:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

where $\mathbf{x}$ and $\mathbf{y}$ are two n-dimensional vectors, and x_i and y_i are the i -th components of vectors $\mathbf{x}$ and $\mathbf{y}$, respectively.

Cosine Distance

Cosine distance measures the cosine of the angle between two vectors and is suitable for clustering data in high-dimensional spaces. The calculation formula is as follows:

$$d_{\cos}(\mathbf{x}, \mathbf{y}) = 1 - \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

where $\mathbf{x} \cdot \mathbf{y}$ represents the dot product of vectors $\mathbf{x}$ and $\mathbf{y}$, and $\|\mathbf{x}\|$ and $\|\mathbf{y}\|$ are the norms (i.e., lengths) of vectors $\mathbf{x}$ and $\mathbf{y}$, respectively.

In this project, since all embedding vectors have the same dimensions and measurement standards, we choose to use Euclidean distance for clustering. The calculation of Euclidean distance is straightforward and intuitive, effectively reflecting the actual spatial distance between vectors. By using Euclidean distance, we can more accurately cluster similar text vectors together, thus achieving a reasonable classification of text data. This not only helps improve the accuracy of clustering but also provides a solid foundation for subsequent analysis and processing.

2.2 LLMs

Large models refer to machine learning models with massive parameters and complex computational structures. These models are typically constructed using deep neural networks, consisting of billions to trillions of parameters. The design goal of large models is to enhance model expressiveness and predictive performance, enabling them to handle more complex tasks and data. Large models find wide applications across various fields including natural language processing, computer vision, speech recognition, and recommendation systems. By training on massive datasets, large models learn intricate patterns and features, exhibiting strong generalization capabilities to make accurate predictions on unseen data.

2.2.1 Two Major Tasks

Text generation tasks are crucial in natural language processing (NLP), aiming to generate coherent natural language text continuously from given in-

puts (prompts, contexts, or other forms of initial information). The primary objective is to create meaningful, grammatically correct text relevant to the given context. Applications include dialogue systems, machine translation, content creation, and story generation.

Text summarization tasks are another important NLP task, aiming to extract key information from a given long text and generate a concise summary. The core goal is to preserve the main content and important details of the original text while significantly reducing its length, facilitating quick understanding and information retrieval. Applications include news summaries, academic paper abstracts, product review summaries, and business report summaries.

There are two types of text summarization tasks. The first is extractive summarization, which directly extracts important sentences or paragraphs from the original text and arranges them in a certain order. This method usually preserves the semantics and style of the original text. The second type is abstractive summarization, which uses language generation models to generate entirely new sentences and paragraphs based on the original content. This approach allows more flexible restructuring of information but requires ensuring the accuracy and coherence of the generated content.

2.2.2 Traditional Methods

Traditional methods for text generation tasks include rule-based systems and statistical language models.

1. Rule-based systems rely on predefined rules and templates, which are simple but lack flexibility and creativity.
2. Statistical language models, such as n-gram models, predict the next word based on the probability of occurrence of the previous n words.

These methods are effective for short text generation but can suffer

from grammar and logic errors.

Traditional methods for text summarization tasks include extractive summarization and abstractive generative summarization.

1. Extractive summarization methods select important sentences from the original text using techniques like TF-IDF, PageRank algorithms, or other statistical features to assess sentence importance.
2. Generative summarization methods based on rules and templates often produce summaries that lack fluency and naturalness.

2.2.3 Large Model Approaches

Large model approaches for text generation tasks include neural network models, particularly recurrent neural networks (RNNs), long short-term memory networks (LSTMs), and Transformer models. Transformer architectures introduce self-attention mechanisms that effectively capture long-distance word dependencies, significantly improving the quality of text generation. Representative models include the GPT series (such as GPT-3, GPT-4).

Neural network language models leverage the powerful modeling capabilities of neural networks to generate language models trained on large-scale corpora. They capture complex context relationships and long-term dependencies, generating high-quality text. There are four main types:

1. Recurrent Neural Networks (RNN): Suitable for modeling sequential data in language with a loop structure that processes each element of the input sequence and maintains context information in hidden layer states.
2. Long Short-Term Memory Networks (LSTM): A specialized RNN that addresses gradient vanishing and exploding problems by introducing

memory units and gating mechanisms (input gate, forget gate, output gate) to control information flow.

3. Gated Recurrent Units (GRU): A simplified version of LSTM that retains most advantages but with lower computational complexity, controlling information flow through update and reset gates.
4. Attention Mechanism: Introducing attention allows models to dynamically select important context information, calculating the importance of each position in the input sequence and weighting the sum accordingly. This significantly improves performance, especially in handling long sequences.

Transformer Model is a neural network architecture based on attention mechanisms that overcome the limitations of RNNs and LSTMs, becoming the mainstream method for text generation tasks. Key features include self-attention mechanisms, multi-head attention, and positional encoding:

1. Self-Attention Mechanism: Calculates the relevance of each position in the input sequence to others, generating a weight matrix to capture global context information. Its strong parallel computing capability is suitable for processing large-scale data.
2. Multi-Head Attention: Uses multiple independent attention heads to capture different feature space information, then concatenates and linearly transforms the results, enhancing the model's representation capability to simultaneously focus on different context features.
3. Positional Encoding: Since Transformer models lack inherent sequence information, positional encoding explicitly introduces positional information. It preserves sequence order information, enhancing the model's ability to process sequence data.

Large model approaches for text summarization tasks include extractive and abstractive generative summarization methods:

1. **Extractive Summarization:** Selects important sentences, phrases, or words directly from the original text to generate summaries without creating new text. Techniques include:
 - (a) **Feature-based methods:** Using specific text features to identify and select key sentences or phrases.
 - (b) **Graph-based methods:** Representing text as a graph, where nodes represent sentences or phrases and edges represent their relationships.
 - (c) **Machine learning methods:** Using supervised or unsupervised learning techniques to predict which parts of the text are most suitable for summarization.
2. **Abstractive Generative Summarization:** Involves generating new text to express summaries rather than extracting content directly from the original text. This method relies on deep learning techniques and large models, such as Transformer architectures:
 - (a) **Seq2Seq models:** Use encoder-decoder structures to encode input text into fixed-length vector representations and then decode to generate summary text.
 - (b) **Transformer models:** Utilize self-attention and multi-head attention mechanisms to capture long-term dependencies and global information in text. They can be fine-tuned after pre-training to achieve higher-quality abstract summaries.
 - (c) **Pre-trained language models:** Such as the GPT series and BERT series, learn language representations through large-scale self-supervised

pre-training and can be further adjusted through fine-tuning for specific summarization requirements.

2.2.4 Principles of Large Models

The basic technological principles of large models include deep neural networks, activation functions, loss functions, optimization algorithms, regularization, and model structures:

1. **Deep Neural Networks (DNN):** Models composed of multiple layers of neurons mapping inputs to outputs through weight matrices. Depth indicates the number of layers, enabling the network to learn complex nonlinear relationships and features.
2. **Activation Functions:** Introduce nonlinear transformations allowing neural networks to learn and represent complex nonlinear mapping relationships. Common functions include sigmoid, tanh, and ReLU (Rectified Linear Unit), each with different performances and advantages in various scenarios.
3. **Loss Functions:** Measure the difference between model predictions and true labels. Optimization algorithms minimize the loss function during training to adjust model parameters for more accurate output predictions. Common loss functions include Mean Squared Error (MSE), Cross-Entropy Loss, chosen based on task requirements.
4. **Optimization Algorithms:** Update model parameters to reduce the value of the loss function. Common algorithms include Gradient Descent, Stochastic Gradient Descent (SGD), Adam optimizer, computing gradients of the loss function and updating parameters in the opposite direction.

5. **Regularization:** Techniques prevent overfitting to training data. Methods like L1 regularization, L2 regularization add penalty terms to constrain model parameter size, enhancing generalization ability.
6. **Model Structures:** Refers to the arrangement and connection of layers in neural networks. Common structures include Fully Connected Networks (FCNs), Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and more complex variants like Transformers. Choice depends on task characteristics and data structure.

These basic principles together form the core of large models. By continuously optimizing and adjusting these technologies, deep learning models adapted for different complex tasks achieve outstanding performance across various application domains.

2.2.5 Common Large Models Comparison

Currently, internationally recognized large models have various types, each with different strengths and characteristics in their respective fields and tasks:

- **GPT (Generative Pre-trained Transformer):** Uses Transformer architecture, focused on generative tasks. Generates text through autoregressive methods, such as dialogue generation, article summarization, etc. GPT series models have made significant advances in generative and language understanding capabilities.
- **T5 (Text-To-Text Transfer Transformer):** Introduces a "text-to-text" framework, unifying multiple NLP tasks' forms. Achieves multi-task learning and transfer learning through standardized task descriptions and pre-training methods. Performs well in various NLP tasks like translation, text classification, etc.

- **BERT (Bidirectional Encoder Representations from Transformers)**: Uses Transformer architecture, enhancing context understanding with bidirectional encoders. Suitable for various downstream tasks of pre-trained models, such as question answering, sentiment analysis, etc. BERT is considered a breakthrough in the NLP field.

模型名称	参数大小 [T]	MMLU [T]	CEval [T]	AGIEval [T]	GSM8K [T]	MATH [T]	BBH [T]	MT Bench [T]	发布者	开源情况
GPT-4o		88.7	/	/	90.5	76.6	/	/		🔒
Claude3-Opus	0.0	86.8	/	/	95.0	60.1	/	9.43		🔒
GPT-4	1750.0	86.4	68.7	/	87.1	42.5	/	9.32		🔒
Llama3-400B-Instruct-InTraining	4000.0	86.1	/	/	94.1	57.8	/	/		🟢
Llama3-400B-InTraining	4000.0	84.8	/	/	/	/	/	/		🟢
Qwen2-72B	727.0	84.2	91.0	/	89.5	51.1	82.4	/		🟢
Gemini-ultra	0.0	83.7	/	/	88.9	53.2	/	/		🔒
Qwen2-72B-Instruct	72.0	82.3	83.8	/	91.1	59.7	/	9.12		🟢
Llama3-70B-Instruct	700.0	82.0	/	/	93.0	50.4	/	/		🟢

Figure 2.1: Comparison Table for Comprehensive Capability Evaluation of Large Models

For our project, we selected the GPT-4o model due to its strong capabilities in logical reasoning, powerful information extraction and inference, efficient text analysis, sentiment analysis, and stance classification.

2.2.6 Prompt Engineering

Prompt Engineering is a technique aimed at designing and optimizing input prompts to guide large-scale language models (such as GPT-3, GPT-4) to generate desired outputs effectively. In natural language processing (NLP), this technique plays a pivotal role, particularly in scenarios with limited training data or specific task requirements. Prompts are textual inputs provided to language models to instruct them on generating specific types of outputs.

The goal of Prompt Engineering is to enhance output quality and accuracy by refining the prompts. It finds broad applications across various domains including text generation, question answering systems, text summarization, translation, and code generation.

The key strategies for effective prompt engineering are as followings:

1. **Clear Task Definition:** Prompts should clearly and specifically describe the task. For instance, instructing the model to generate a summary on a given topic.
2. **Contextual Information:** Providing adequate contextual information within prompts helps the model better understand the task requirements. For example, offering background information before posing questions.
3. **Example-Based Prompts:** Using examples or templates as prompts aids the model in producing outputs similar to provided examples. For instance, presenting several question-answer pairs and expecting the model to generate similar responses.
4. **Structured Formats:** Using structured formats such as lists or headers assists in generating well-organized outputs. For example, instructing the model to generate a list containing multiple items.
5. **Iterative Optimization:** Continuous refinement of prompts through iterative testing and adjustments based on model output observations. This iterative process helps in optimizing prompts to achieve optimal results across various tasks and models.

Prompt Engineering leverages language models to generate high-quality outputs by guiding them effectively through well-designed prompts. By refining and optimizing prompts, users can steer models to produce text content

that meets specific requirements. In practice, ongoing testing and optimization of prompts are crucial to adapt to different task demands and model capabilities.

2.3 Kity Minder

KityMinder is a sophisticated online mind mapping tool developed by Baidu's FEX team, designed with developers in mind. Built on the powerful Kity vector graphics library, KityMinder offers extensive capabilities for data visualization and basic editing through its core module, `kityminder-core`.

KityMinder-core provides a robust foundation for developing custom mind mapping solutions. It supports various formats for data import and export, including JSON, text, Markdown, SVG, and PNG, making it highly versatile for integration into different applications. This flexibility allows developers to tailor the tool to specific needs, whether for personal projects or enterprise-level applications.

The KityMinder Editor enhances the core's functionality by incorporating AngularJS, providing a rich user interface and efficient input mechanisms such as hotbox components. This setup facilitates rapid development and a seamless user experience, making it easier to implement complex features and improve overall usability.

Developers can leverage KityMinder's comprehensive API and detailed documentation to integrate and extend its features effortlessly. The tool supports major browsers, including Chrome, Firefox, Safari, and IE 10+, ensuring broad compatibility and a consistent experience across different platforms.

For those looking to contribute or customize further, KityMinder's open-source nature encourages community involvement and collaboration. The project is hosted on GitHub, where developers can fork the repository,

contribute to its development, and share improvements with the broader community.

2.4 Sentiment Analysis

2.4.1 Introduction to Sentiment Analysis

Text sentiment recognition, also known as sentiment analysis, is a sub-field of Natural Language Processing (NLP) that aims to identify and extract subjective information from text data, determining whether the expressed opinions are positive, negative, or neutral. This task is crucial for various applications, such as analyzing customer feedback, monitoring social media, and enhancing decision-making processes for businesses and governments.

Sentiment analysis involves multiple steps, including data preprocessing, feature extraction, and classification, to handle the unstructured nature of text data and the complexity of human language [8][9]. One of the primary challenges in sentiment analysis is accurately interpreting the context and nuances of language, such as sarcasm, negation, and domain-specific terminology, which can significantly alter the sentiment conveyed [9].

Techniques for sentiment analysis can be broadly categorized into machine learning approaches, which rely on labeled datasets and algorithms to learn patterns, and lexicon-based approaches, which use predefined dictionaries of sentiment-laden words [8]. Recent advancements have also seen the integration of deep learning methods to improve the performance and accuracy of sentiment analysis systems [9].

Despite these advancements, there remain significant challenges, such as handling low-resource languages and code-mixed data, which require ongoing research and innovation to address effectively [8][9].

2.4.2 Multilingual Sentiment Analysis Model

The bert-base-multilingual-uncased-sentiment model by NLP Town is a fine-tuned BERT model for sentiment analysis on product reviews in six languages: English, Dutch, German, French, Spanish, and Italian. It predicts the sentiment of a review on a scale from 1 to 5 stars. The model has been trained on a substantial dataset of product reviews and has demonstrated high accuracy, particularly in distinguishing reviews with star ratings within one star of the human reviewer.

Chapter 3: Architecture Design

In the development of any large-scale system, the overall architecture serves as its backbone, determining the system’s functionality, stability, performance, and ease of maintenance. A well-designed architecture not only ensures efficient operation but also saves significant time and effort in future expansions and maintenance.

Cocoon Breaker is a complex web application with a Python-based backend. Its design aims to provide users with a concise and aesthetically pleasing user interface, while also ensuring that developers find the codebase easy to read, maintain, and extend through convenient function libraries.

- **User Interface:** The user interface of Cocoon Breaker is designed to be simple and intuitive, aiming to deliver an excellent user experience. Navigation and functional operations are straightforward, minimizing the learning curve and operational complexity for users.
- **For Developer:** Django is a high-level Python web framework known for its “batteries-included” philosophy, offering features like authentication, admin interface, template engine, ORM, and more out of the box. This simplifies web application development significantly, aligning with Django’s principles of “rapid development” and “DRY (Don’t Repeat Yourself).” In contrast, Flask is a lightweight framework that provides flexibility, requiring developers to integrate various extensions themselves, making it ideal for small projects or highly customizable applications. Ruby on Rails, a popular Ruby framework, emphasizes rapid

development and convention over configuration. Laravel, a leading PHP framework, is praised for its elegant syntax and robust features, though it relies on PHP-specific characteristics. Django’s strengths include a comprehensive set of built-in features, reducing developer workload, and a large, active community offering extensive third-party plugins and documentation. Django also excels in security, with built-in protections against common vulnerabilities like SQL injection, XSS, and CSRF. Python’s advantages in backend development are clear. The language is easy to use and highly readable, making development more efficient. Python’s simple syntax allows developers to focus on problem-solving rather than complex code. Additionally, Python’s rich libraries and frameworks cater to various needs, from web development to data analysis and AI. With modular design and extensive third-party support, Python applications can easily expand their functionalities, including user-friendly machine learning frameworks.

Given the complexity and interdependence of Cocoon Breaker’s functionalities, we implemented an asynchronous design to enhance system responsiveness and user experience. This approach not only ensures the correctness of operations but also significantly reduces user wait times. For instance, while handling user requests, the backend can perform multiple operations concurrently rather than sequentially. This parallel processing greatly improves system performance.

To ensure data safety and stability when multiple users access the system simultaneously, Cocoon Breaker uses the browser’s session ID to achieve user data isolation. This design effectively prevents data and processing conflicts, ensuring stable operation even under high concurrency conditions.

Cocoon Breaker is deployed on a server and has undergone rigorous testing and verification to ensure stable and efficient operation in real-world

usage. Through a series of validation steps, we confirmed that all system functionalities work correctly and meet user requirements.

3.1 Overall Architecture

Our program leverages a diverse array of technologies that seamlessly integrate to create a robust and comprehensive workflow, each playing a crucial role in ensuring the system’s overall efficiency and effectiveness. The table below provides a detailed summary of these technologies and their specific applications within our workflow:

Task	Technology	Usage
Backend	Django	Local/Server Deployment
Online Search	Google API	API Service
Video Info Scraping	Bilibili API	API Service
Web Scraping	BeautifulSoup	—
Large Language Model (LLM)	GPT-4o	API Service
Sentiment Analysis	bert-base-multilingual-uncased-sentiment	Local Deployment
Embedding	bert-base-multilingual-cased	Local Deployment

Table 3.1: Summary of Technologies Used in Our Program

Specifically, we use Django as our backend framework. Django’s efficiency and flexibility enable rapid development and deployment of applications. We deploy Django both locally and on servers to ensure system stability and scalability. For online search functionality, we use the Google API. The Google API provides powerful search capabilities and rich data sources; by invoking its API service, we can quickly obtain the necessary information and integrate it into our workflow.

To obtain video information, we use the Bilibili API. By calling the API

service provided by Bilibili, we can retrieve and process a large amount of video data. This is crucial for analyzing video content and extracting key information. For web scraping, we chose BeautifulSoup, a simple and easy-to-use Python library suitable for handling HTML and XML files. BeautifulSoup allows us to parse web content and extract valuable information.

We have integrated GPT-4o as the large language model (LLM) in our system. By invoking its API service, we can perform natural language processing tasks such as text generation and question answering. This makes our system perform exceptionally well in handling complex texts. For sentiment analysis, we use the bert-base-multilingual-uncased-sentiment model. This model can identify sentiments in multiple languages and is deployed locally to ensure efficient and secure analysis. For embedding tasks, we use the bert-base-multilingual-cased model. This model generates high-quality embeddings for texts in multiple languages and can quickly process large amounts of text data when deployed locally.

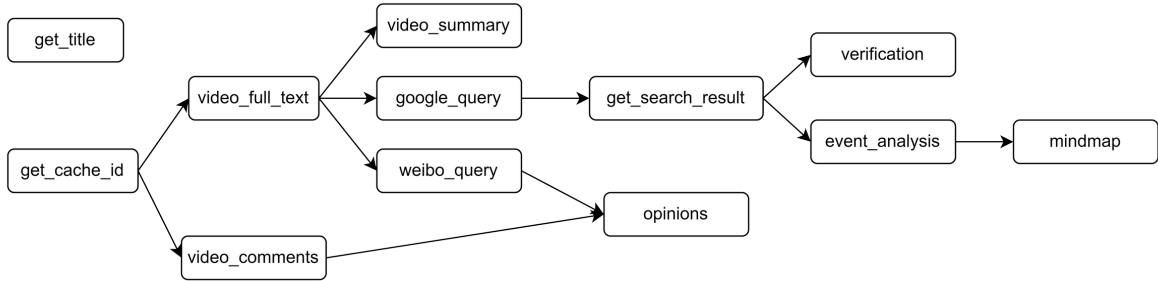


Figure 3.1: Directed Acyclic Graph (DAG) of Task Modules

Additionally, in our program design, we particularly focus on using asynchronous processing methods. By organizing the various subtasks into a directed acyclic graph (DAG), as shown in Figure 3.1, we can clearly analyze their dependencies. In practice, for example in online search, we use asynchronous methods to concurrently scrape a large number of web pages, significantly improving efficiency. Asynchronous processing also reduces sys-

tem response time, making the entire workflow more smooth and efficient.

3.2 Web Plugin

Bilibili Analyzer is a browser extension designed specifically for Google Chrome-based browsers. It aims to help users analyze the Bilibili video they are currently viewing and automatically navigate to a related report page. Once installed and enabled, users will see a **Bilibili Analyzer** icon in the browser toolbar. Clicking this icon will open a small window with a prominent “Start Analysis” button. This simple and clear design ensures a convenient and efficient user experience.



Figure 3.2: Screenshot of the plugin’s user interface

When the user clicks the “Start Analysis” button, the extension first calls the `chrome.tabs.query` API to get the URL of the current active tab and uses a regular expression to validate that URL, ensuring it matches the format of a Bilibili video page. Specifically, the regular expression matches the URL’s structure to identify a legitimate video page. If the current page is a valid Bilibili video page, the extension extracts the video’s BV number, which is the unique identifier for Bilibili videos. The BV number is a crucial parameter used to uniquely identify each video on the Bilibili platform. By extracting the BV number, the extension can accurately locate the specific video content.

Next, the extension constructs a new report page URL, which includes the extracted BV number and points to a pre-configured analysis report server address. This server address is used to generate and display a detailed

analysis report of the video. The extension then uses the `chrome.tabs.create` API to open this report page in a new tab, allowing the user to directly view the detailed analysis report of the video.

If the extension detects that the current page is not a valid Bilibili video page, it will pop up a message box to inform the user that the current page is invalid and suggest using the extension on a proper Bilibili video page. This design not only enhances the user experience but also ensures that the analysis function is only run on appropriate pages, avoiding unnecessary operations and confusion.

The development of the **Bilibili Analyzer** extension follows the Chrome Extension **Manifest V3** specification. **Manifest V3** provides a clear structure and security guarantees for the functionality and permissions of extensions. **Manifest V3** introduces a series of new features and improvements aimed at enhancing the security, privacy protection, and performance of extensions. The extension primarily relies on the `chrome.tabs.query` API to get the URL of the current active tab and uses the `chrome.tabs.create` API to open the report page in a new tab. Through these APIs, the extension achieves efficient management and operation of browser tabs, ensuring a smooth user experience during use.

3.3 Frontend and Backend Design

3.3.1 Frontend Design

The frontend interface serves as the bridge for user interaction with the application, directly impacting user experience, brand image, and the success of the application. To enhance the usability of our project, we have designed the frontend interface with a focus on visual appeal and ease of use.



Figure 3.3: Application Main Interface1

The background image features a high-tech, predominantly blue and black color scheme. Blinking blue lights symbolize the rapid transmission of information, suggesting breaking through information barriers, aligning with the theme of “Info Cocoon Breaker.” The project’s theme is prominently displayed in the center with the largest font size. An input box allows users to paste the link of the video they wish to analyze. Above the input box, the prompt “Currently revising video:” and the text in the input box “Please copy the video link here” guide users on the required action. Below the input box is a “Start Analysis” button, which turns yellow when hovered over, indicating its clickable nature.



Figure 3.4: Application Main Interface2

Upon clicking the “Start Analysis” button, it disappears, signaling to users that a video is currently being analyzed and preventing them from initiating another analysis. Below the input box, a progress bar and the message “Analysis in progress...” inform users about the ongoing system activity. The progress bar displays the current status of the video analysis, providing real-time feedback on the progress of the task, reducing user anxiety and uncertainty, and keeping them informed of the system’s operations. Once the progress bar completes loading, the page redirects to the analysis results interface.

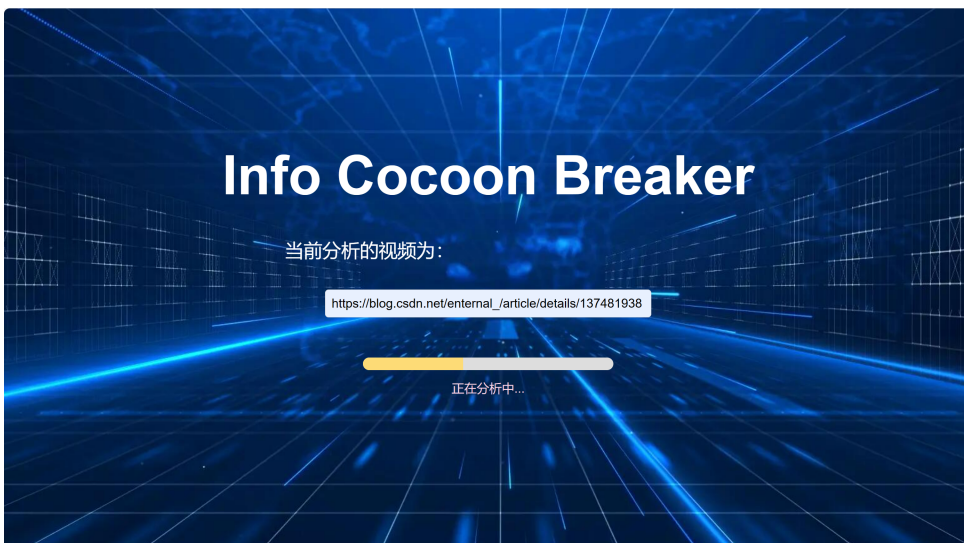


Figure 3.5: Application Main Interface3

At the top right of the page is the project's name, clickable to return to the main interface. Positioned slightly below on the left is the title of the currently analyzed video, clickable to return to the original video interface. The analysis results page comprises four modules: "Video Summary," "Fact Finding," "Timeline Overview," and "Multiple Perspectives." Each module is divided into two sections: the left section features an index directory, while the right section displays detailed content for each module of the analysis results. Structured analysis results transmitted from the backend to the frontend automatically generate an index displayed on the left, distinguishing headings with font sizes and bold text to maintain clarity and visual appeal, making it easier to extract key points from lengthy text. Clicking on the title in the left index section automatically navigates to the corresponding module's display on the right. The "Timeline Overview" and "Multiple Perspectives" modules also feature relevant result images. The entire interface is clean, elegant, with clear functionality, and aesthetically pleasing, demonstrating thoughtful design.



Figure 3.6: interface of video summary

3.3.2 Backend Design

Our backend system is implemented using the Django framework, which has been extensively detailed in the previous sections outlining its many advantages. The backend design adheres to two key principles: asynchronous operation and modularity. Modularity forms the foundation of the entire system design. To achieve this principle, we have divided the backend system into 11 basic modules based on the characteristics of the tasks and practical considerations. These modules are functionally independent, with each module implemented by a separate class, ensuring high cohesion and low coupling of the code.

Additionally, these modules have sequential dependencies in data processing. Therefore, we have organized the tasks based on figure 3.1 and determined an asynchronous design scheme. Data transmission is handled through Django’s built-in caching mechanism, with session IDs used to ensure isolation between different users’ data. The asynchronous design not only improves the system’s execution efficiency but also significantly reduces resource usage and response time, thereby enhancing the user experience.

During the collaborative development process, each class was developed and tested independently, and the workflow functioned correctly upon the final merge. This design approach allows for the addition of new functional modules or the optimization and expansion of existing ones without affecting the overall system, thus ensuring the long-term stable development of the system. In summary, the principles of asynchronous operation and modularity complement each other, making our backend system not only highly performant but also exceptionally extensible and flexible.

Below is an introduction to these 11 modules, with more detailed descriptions provided in the subsequent sections:

- **GoogleQuery:** The GoogleQuery class is designed to read text from

the cache and generate a query suitable for Google search using OpenAI's GPT model. The input to this class is a `cache_key`, and the output is the generated Google query terms. By reading the complete text stored in the cache, the class uses the GPT model to extract keywords from the text and stores the resulting query back into the cache.

- **EventVeri:** The EventVeri class is used to verify the authenticity of events. The input is a `cache_key`, and the output is a factual analysis report of the event. This class reads the full text and search results from the cache and uses OpenAI's GPT model to generate a detailed factual analysis report, which is then returned as the output.
- **EventAnalyzer:** The EventAnalyzer class is responsible for analyzing events and generating an HTML-formatted analysis report. The input to this class is a `cache_key`, and the output is an HTML-formatted event analysis report. By reading the complete text and search results from the cache, the class uses the GPT model to generate a detailed event analysis report, which is then stored back into the cache.
- **AnalysisMindmap:** The AnalysisMindmap class converts event analysis reports into mind map format. The input is a `cache_key`, and the output is the HTML content of the mind map. This class reads the event analysis report from the cache and uses OpenAI's GPT model to generate a Markdown-formatted mind map, which is then converted to HTML format and returned.
- **WeiboQuery:** The WeiboQuery class is designed to generate concise queries suitable for Weibo hot searches. The input is a `cache_key`, and the output is the query terms formatted for Weibo. By reading the complete text from the cache, the class uses the GPT model to generate concise Weibo hot search queries and stores the resulting queries back

into the cache.

- **VideoSummary:** The VideoSummary class processes video text data. The input is a `cache_key`, and the output is an HTML-formatted summary of the video content. By using the `cache_key`, this class reads the full text of the video from the cache and employs OpenAI's GPT model to generate a detailed summary of the video content. The summarized content is presented in a clear HTML format for easy viewing and use.
- **VideoInfo:** The VideoInfo class retrieves information about Bilibili videos. The input parameters are `bv` (the BV number of the Bilibili video) and `cache_key`, and the output is the video's title. This class uses the BV number to request video page data from Bilibili's API, extracting the video's title, description, and tags, and downloading the audio file. The audio file is then converted to WAV format for subsequent processing and analysis.
- **VideoFullText:** The VideoFullText class is designed to transcribe the audio file of a video into complete text. The inputs to this class are `bv` (the BV number of a Bilibili video) and `cache_key`, and the output is the full text of the video. By invoking the iFlytek API, the class uploads and merges audio file segments to obtain the transcribed text result, which is then stored in the cache.
- **VideoComments:** The VideoComments class is used to retrieve comments and danmu (bullet screen comments) data from Bilibili videos. The inputs are `bv` (the BV number of a Bilibili video) and `cache_key`, and the output is a CSV file containing the comments and danmu data. By using the BV number, the class fetches the video's CID from Bilibili's API, then requests the danmu XML data and comments JSON data. After parsing, the data is stored as a CSV file and the result is

saved to the cache.

- **SearchResult:** The SearchResult class performs Google searches and retrieves the page content of the search results. The input is cache_key, and the output is the text content of the search results. By reading the Google search query terms from the cache using the cache_key, the class utilizes the Google Search API to obtain search results. It then uses multithreading to concurrently request the content of each search result page, extracts the text content, aggregates it, and stores the result back into the cache.
- **VideoAnalyzer:** The VideoAnalyzer class analyzes video content and generates an HTML-formatted report. The input is cache_key, and the output is an HTML-formatted video analysis report. By reading the complete text of the video and the search results from the cache, the class uses OpenAI's GPT model to generate a detailed video analysis report, which is then stored in the cache.

3.4 Video Processing

3.4.1 Video Information Retrieval

To obtain video title information from Bilibili, we need to access the webpage or API through request headers, simulating real user behavior to avoid being identified as bots or crawlers by the server, thereby acquiring more accurate and comprehensive data.

Request Headers

1. User-Agent

- **Function:** The User-Agent header tells the server information about the client's software (e.g., browser type, operating system,

etc.).

- **Reason:** Many websites determine whether a request is from a browser or a crawler based on the User-Agent header. If the User-Agent is not set correctly, the server might return different content or block access altogether.

2. Referer

- **Function:** The Referer header contains the URL of the page that initiated the current request.
- **Reason:** Some websites check the Referer header to ensure the request comes from a page within the site, rather than directly through an API or script. Setting the Referer header can increase the credibility of the request and reduce the likelihood of being denied.

3. Cookie

- **Function:** The Cookie header contains the user's session information and other state information.
- **Reason:** Many websites use cookies to store the user's login status, preferences, etc. By setting the correct cookies, we can simulate a logged-in state, allowing access to user-specific data or restricted content.

Retrieve a Single Video Title

To retrieve the video title, first construct the video page URL using the BV number with the code:

```
url=f'https://www.bilibili.com/video/{self.bv}'
```

Next, send an HTTP GET request to this URL using:

```

def get_single_audio_title(self):
    url = f'https://www.bilibili.com/video/{self.bv}'
    response = requests.get(url=url, headers=self.headers)
    title = re.findall(pattern: '<h1 data-title="(.*?)" title="(.*?)" class="video-title spe
    response.text)[0][0]
    return title
1 个用法
def get_collection_audio_title(self, headers):
    print("请输入B站合集序号: ")
    a = input()
    i = 1
    url = f'https://api.bilibili.com/x/web-interface/wbi/view/detail?platform=web&bvid={se
    response = requests.get(url=url, headers=self.headers)
    json_data = response.json()
    pages = json_data['data']['View']['pages']
    for page in pages:
        if i < int(a):
            i += 1
            continue
        title = page['part']
    return title

```

Figure 3.7: Core code of retrieving the title from video

```
response=requests.get(url=url,headers=self.headers)
```

Then, parse the HTML response to extract the video title by using the regular expression:

```

<h1 data-title="(.)" title="(.)"
class="video-title special-text-indent" data-v-5120f6b9>

```

The extracted title can be obtained with:

```

title=re.findall('<h1data-title="(.)"title="(.)"class=
"video-titlespecial-text-indent"data-v-5120f6b9>',response.
text)[0][0]

```

Retrieve Titles of Videos in a Collection

To scrape a specific video from the collection, start by prompting the user to enter the index of the video they want to scrape. Then, construct the API request URL based on the BVID and other necessary parameters. Next, send an HTTP GET request to the constructed URL. Upon receiving the response, use:

```
response.json()
```

to parse the response content into a JSON object. Finally, extract the video title from the JSON object with the line:

```
title=response.json()['data']['View']['pages'][int(a)-1]
['part']
```

3.4.2 Video Text Extraction

The main functionality of the code includes downloading the audio file of a Bilibili video, using iFlytek Spark to convert the audio to text, and saving the result as a `.txt` file. First, the video title is obtained and cleaned of any illegal characters, similar to the code for obtaining video title information mentioned earlier. This process includes the cleaning of illegal characters.

Retrieve the Audio File of a Single Bilibili Video

To get the audio file of a single Bilibili video, start by calling the `get_single_audio_title()` method to get the cleaned video title. Construct the video page URL and use the `requests` library to send a GET request to retrieve the page content. Use the regular expression `re.findall()` to extract video play information from the page and parse the extracted JSON data into a dictionary format (`json_data`). Extract the audio URL from `json_data`, typically found in the `dash` field's `audio` array's first element's `baseUrl` field. Next, construct the target file path (`target_path`), usually saved in the `video` folder, with the file name using the video title. Use the `convert_audio_to_wav()` method to download the audio file, convert it to `.wav` format, and save it to `target_path`. If the `video` folder does not exist, create it before saving the file.

Retrieve Audio Files of a Bilibili Video Collection

To retrieve audio files from a Bilibili video collection, start by getting the video title and CID using the `get_collection_audio_title()` method, which returns the cleaned video title (`cleaned_title`), video CID (`cid`), and some other unused variables.

Next, construct the API request URL using the video's BV number and the retrieved CID. This URL is typically used to obtain detailed playback information of the video. For example, the URL might look like

```
https://api.bilibili.com/x/player/wbi/playurl?avid=571565015&
bvid=\{self.bv\}&cid=\{cid\}
```

After that, extract the audio URL from `link_data`, usually found in the `dash` field's `audio` array's first element's `baseUrl` field

```
audio_url=link_data['data']['dash']['audio'][0]['baseUrl']
]
```

Finally, construct the target file path (`target_path`), typically saved in the `video` folder with the file name derived from the video title. Ensure the `video` folder exists by using `os.makedirs('video', exist_ok=True)`. Call the `convert_audio_to_wav()` method to download the audio file, convert it to `.wav` format, and save it to `target_path`. If the `video` folder does not exist, create it before saving the file.

Preparing for Audio-to-Text Conversion

The code defines various API interfaces and file chunk sizes, setting necessary parameters such as application ID and key during initialization. To generate parameters needed for sending requests to the iFlytek Spark API, the code performs several steps:

```

def get_single_audio(self):
    cleaned_title = self.get_single_audio_title()
    url = f'https://www.bilibili.com/video/{self.bv}'
    response = requests.get(url=url, headers=self.headers)
    play_info = re.findall(pattern: '<script>>window.__playinfo__=(.*?)</script>', response.text)[0]
    json_data = json.loads(play_info)
    audio_url = json_data['data']['dash']['audio'][0]['baseUrl']
    target_path = os.path.join('video', f'{cleaned_title}.wav')
    os.makedirs(name='video', exist_ok=True)
    self.convert_audio_to_wav(audio_url, target_path)
    return cleaned_title

1 个用法
def get_collection_audio(self):
    cleaned_title, cid, a = self.get_collection_audio_title()
    link = f'https://api.bilibili.com/x/player/wbi/playurl?avid=571565015&bvid={self.bv}&cid={cid}'
    link_data = requests.get(url=link, headers=self.headers).json()
    audio_url = link_data['data']['dash']['audio'][0]['baseUrl']
    target_path = os.path.join('video', f'{cleaned_title}.wav')
    os.makedirs(name='video', exist_ok=True)
    self.convert_audio_to_wav(audio_url, target_path)
    return cleaned_title

```

Figure 3.8: Core code of retrieving audio from video

1. **Generate Timestamp:** A timestamp is generated to ensure the request is current.
2. **Generate MD5 Signature:** An MD5 signature is created for data integrity.
3. **Generate HMAC Signature:** An HMAC signature is produced for secure authentication.

Next, the code retrieves the file length and file name. Based on different APIs, it generates the corresponding parameter dictionary for the following interfaces:

- Preparation Interface
- Upload Interface
- Merge Interface
- Progress and Result Interface

Handling HTTP POST Requests

To send an HTTP POST request to the specified iFlytek Spark API and handle the response, the code performs the following steps: it sends the request, handles the response, and checks the response result to ensure the request was successful.

Uploading Audio Files

To upload an audio file to iFlytek's Speech Recognition Service, the file is divided into chunks to improve success rate and efficiency, with each chunk sized at 100MB. The process involves opening the audio file in binary mode, reading the file content by the specified chunk size, obtaining the ID of the next chunk, and calling the API interface to upload each chunk until the entire file is uploaded.

Merge Audio Files

Merge all uploaded chunks into a complete audio file on the server by calling the API interface to notify the server to merge all chunks into one file, which is essential for subsequent speech recognition processes.

Get Conversion Progress and Result

Regularly query the server for task progress until the recognition task is complete. Periodically call the progress query interface to check the task status. Once the task is complete, call the result retrieval interface to obtain the recognition result and save it as a text file.

3.4.3 Video Comment and Danmu Crawling

The main functionality of the code includes initializing functions to obtain Bilibili video's bullet comment (danmaku) data and comment data. It

then saves this data to a CSV file and caches the retrieved information for later use.

Obtain Danmaku XML Data

To obtain the danmaku XML data, the process begins by fetching the video's `cid` through the Bilibili API using the BV number. This is achieved by constructing a request URL formatted with the `cid` obtained from the `_get_cid_from_bv()` method. Following this, an HTTP GET request is sent to retrieve the data. The response encoding is set to UTF-8 to ensure accurate handling of Chinese and special characters. Upon a successful request, the response text, which contains the danmaku XML data, is returned.

Parse Danmaku XML Data

First, parse the XML string into an XML tree to facilitate structured access to the data. Next, traverse all danmaku nodes within the XML tree to systematically extract the send time, sender ID, and danmaku content. Each extracted danmaku entry is formatted appropriately and then added to a list for further processing or storage. This systematic extraction ensures that all relevant danmaku data is captured efficiently and accurately for subsequent analysis or usage.

Entry Point for Obtaining Danmaku Data

The entry point for obtaining danmaku data involves calling the methods of `_get_cid_from_bv()`, `_get_danmaku_xml()`, and `_parse_danmaku_xml()` sequentially to retrieve the video's CID, fetch the danmaku XML data, and parse it to extract relevant danmaku information. Subsequently, the parsed danmaku data is saved into a CSV file and cached for future use. Relevant code as follows:

```

def get_bullet_comments(self):
    cid = self._get_cid_from_bv(self.bv)
    if cid:
        print(f"获取到的 cid: {cid}")
        xml_data = self._get_danmaku_xml(cid)
        if xml_data:
            danmakus = self._parse_danmaku_xml(xml_data)
            if danmakus:
                b_comments = pd.DataFrame(danmakus, columns=['danmakus_time', 'user_id', 'danmakus'])
                b_comments.to_csv(path_or_buf='danmakus.csv', index=False, encoding='utf-8-sig')
                print("弹幕数据已保存到 'danmakus.csv'")
                self.write_cache(key=f'{self.bv}_弹幕_{self.cache_key}', b_comments)
            else:
                print("没有获取到弹幕数据")
        else:
            print("获取弹幕 XML 数据失败")
    else:
        print('获取视频的 cid 失败')

```

Obtain the Video's OID

First, we need to get the video's OID. This is done by requesting the Bilibili API using the BV number to get the video's AV number (OID).

Obtain Comment Data for a Specified Bilibili Video

To obtain comment for a specified Bilibili video, call `get_oid_from_bv` method to get the video's OID. Then, construct the API request URL and send the request.

Initialize a comment data list and loop to get all comment data.

- In each loop iteration: Extract the comment list `result` from the current page's data.
- Use a `try` block to iterate through each comment item in the comment list, extract the comment content and time, and store them in the `comments_data` list.
- If a comment item contains replies, iterate through the reply list, extract the reply content and time, and store them in the `comments_data` list.
- Construct the API request URL for the next page, send the request, and update `next_page` and `is_end`.

- If a `TypeError` exception occurs during processing, print the error message and stop the loop.

Finally, save the comment data as a CSV file and cache it. Relevant code as follows:

```
def get_comments(self):
    oid = self.get_oid_from_bv(self.bv)
    print(oid)
    url = f"https://api.bilibili.com/x/v2/reply/main?jsonp=jsonp&next=0&type=1&oid={oid}&mode=3&plat=1"
    response = requests.get(url=url, headers=self.headers).json()
    next_page = response['data']['cursor']['next']
    is_end = response['data']['cursor']['is_end']

    comments_data = [] # 用于存储评论数据的列表

    while not is_end:
        result = response['data']['replies']
        try:
            for item in result:
                user = item['member']['uname']
                comment = item['content']['message']
                comment_time = time.strftime(format: "%Y-%m-%d %H:%M:%S", time.localtime(item['ctime']))
                comments_data.append({'comment_time': comment_time, 'user': user, 'comment': comment})

                if 'replies' in item:
                    for index in item['replies']:
                        reply_user = index['member']['uname']
                        reply_comment = index['content']['message']
                        reply_time = time.strftime(format: "%Y-%m-%d %H:%M:%S", time.localtime(index['ctime']))
                        comments_data.append(
                            {'comment_time': reply_time, 'user': reply_user, 'comment': reply_comment})

            url = f"https://api.bilibili.com/x/v2/reply/main?jsonp=jsonp&next={next_page}&type=1&oid={oid}&mode=3&plat=1"
            response = requests.get(url=url, headers=self.headers).json()
            next_page = response['data']['cursor']['next']
            is_end = response['data']['cursor']['is_end']

        if is_end:
            break
```

3.5 Online Search

In our project, we have developed a Python class named `SearchResult`, which integrates multiple technologies and open-source libraries to perform bilingual Google searches, text translation, and web content extraction. This class achieves the following core functionalities:

Bilingual Google Search Functionality

Firstly, we utilize the Google Custom Search API to implement bilingual search capabilities. Below is the relevant code snippet and explanation:

```
def google_search_use_chinese_and_english(self, **kwargs):
    query = self.google_query
    query_trans = self.translate_query(query)
    print('query and tras query:', query, query_trans)

    result1 = self.google_search(query)
    result2 = self.google_search(query_trans)
    items1 = result1.get('items', [])
    items2 = result2.get('items', [])
    result1 = [[item['title'], item['link']] for item in items1]
    result2 = [[item['title'], item['link']] for item in items2]
```

This code demonstrates how the method is invoked to handle simultaneous Chinese and English queries, utilizing the `google_search` method to retrieve search results and merge them.

AI Translation Support

Secondly, we leverage OpenAI's GPT model to achieve automatic translation between Chinese and English. Below is the corresponding code and explanation:

```
# 使用GPT进行中英互译用于中英文搜索
def translate_query(self, content, model = "gpt-4o"):
    completion = self.client.chat.completions.create(
        model=model,
        messages=[
            {
                "role": "system",
                "content": "你的任务是中英互译，如果接收英文则翻译为中文；如果接收中文则翻译为英文。不需"
            }, {
                "role": "user",
                "content": content
            }
        ]
    )
    return completion.choices[0].message.content if completion.choices[0].message.content
```

This snippet illustrates how the `translate_query` method is called to perform Chinese-English translation using OpenAI's GPT model, supporting users' search needs in different language environments.

Web Content Extraction and Concurrent Processing

Lastly, we implement web content extraction using the BeautifulSoup library and accelerate the processing of search results using Python's concur-

rency features. Below is the relevant code and explanation:

```
def result_of_google(self, results):
    all_text_content = '搜索结果: \n'
    with concurrent.futures.ThreadPoolExecutor(max_workers=10) as executor:
        # 创建一个future到get_page_text的映射
        future_to_page = {
            executor.submit(self.get_page_text, result[1]): result
            for result in results
        }
        for future in concurrent.futures.as_completed(future_to_page):
            page_text = future.result()
            if page_text:
                result = future_to_page[future]
                title = result[0]
                all_text_content += f'Title: {title}\n\n{page_text}\n\n{"="*80}\n\n'
    return all_text_content
```

This code demonstrates how `result_of_google` method utilizes concurrent processing and cache management to fetch detailed text content of search results and present it comprehensively.

By combining these functionalities, the `SearchResult` class not only enhances the efficiency and accuracy of information retrieval but also provides technical support and demonstrations for research involving multilingual content and complex network environments.

3.6 GPT Analysis

3.6.1 Video Summary

Define a class named `VideoSummary`, which is used to read the complete text of the video from the cache key and generate a summary of the video content using OpenAI's GPT model.

In the initialization function, the `cache_key` is obtained as a parameter. The `read_cache` function is used to read the complete text from the cache. The first parameter is `cache_key`, and the second parameter is the content associated with the key 'full_text' in the cache (which reads the complete text of the video). Print separators, class names, cache keys, and

read complete text for debugging and inspection of class initialization.

An instance method `get_video_summary` is used to generate a summary of the video content. It calls OpenAI's GPT model, specifying the model to call as `gpt-4o` and passing two messages: the first message is a system message providing instructions to the model to generate a summary in a specified format; the second message is user information providing the complete text content to be summarized. Finally, the function returns the summary generated by the model.

```
class VideoSummary(object):
    def __init__(self, cache_key):
        self.cache_key = cache_key
        # 通过此方法获取full_text
        self.full_text = read_cache(self.cache_key, 'full_text')
        print('=====')
        print(f'{self.__class__.__name__}类初始化情况')
        print(self.cache_key)
        print(self.full_text)
        print('=====')

    def get_video_summary(self):
        completion = client.chat.completions.create(
            model="gpt-4o",
            messages=[
                {"role": "system", "content":
                    r"下面将给出视频文案，请你全面详细总结视频内容，分点明确，不要出现下面格式不存在的标签(如"
                {"role": "user", "content": self.full_text}
            ]
        )
        return completion.choices[0].message.content if completion.choices[0].message.content
```

Figure 3.9: Viedo Summarization Code

Improvement of instructions. When designing instructions for calling GPT to generate content, enter basic instructions according to requirements, the effect is general. The prompt word engineering later adjusts the instructions according to the error of the output results. The return effect is significantly improved.

3.6.2 Extracting Search Keywords

By using OpenAI's GPT model, we generate short summaries suitable for Weibo hot searches and keywords suitable for Google searches from the given text. To achieve this, we define two classes: `WeiboQuery` and `GoogleQuery`. These classes use OpenAI's GPT-4o model to generate corresponding query summaries and keywords through different prompts.

- **WeiboQuery Class:** Responsible for generating short summaries in the format of Weibo hot searches.
- **GoogleQuery Class:** Responsible for extracting keywords suitable for Google searches.

The main approach is to guide the GPT model to generate the required output through prompts.

3.6.3 Event Analysis

The class `EventAnalyzer` is designed to perform event analysis and generate an HTML-formatted analysis report using the OpenAI GPT-4o model.

Key Functionalities:

Initialization (`__init__` method): Initializes the class instance with a given `cache_key`, retrieves necessary data from the cache (`full_text` and `s_result`), and creates an instance of the OpenAI client for natural language processing tasks.

Explanation:

- **`cache_key`:** Stores the identifier for retrieving data from the cache.
- **`full_text` and `s_result`:** Store the complete text and search results respectively retrieved from the cache using `read_cache`.

```
def __init__(self, cache_key) -> None:
    self.cache_key = cache_key
    self.full_text = read_cache(self.cache_key, 'full_text')
    self.s_result = read_cache(cache_key, 's_result')
    self.client = OpenAI()
    print('=====')
    print(f'{self.__class__.__name__}类初始化情况')
    print(self.cache_key)
    print(self.s_result[:100])
    print('=====')
```

- **client:** Instance of the OpenAI client for accessing the GPT-4 model.
- Prints initialization details including class name, cache key, and a snippet of the retrieved search results (s_result).

Generating HTML Analysis (analysis_to_html method): Uses the GPT-4o model to generate a detailed HTML-formatted analysis report based on the combined full_text and s_result.

```
def analysis_to_html(self, s_result):
    completion = self.client.chat.completions.create(
        model="gpt-4o",
        messages=[
            {"role": "system", "content":
                r"下面将给出一件事的所有相关信息，请你全面详细总结该事件的来龙去脉以及其他各个方面"}
            {"role": "user", "content": s_result}
        ]
    )
    return completion.choices[0].message.content if completion.choices[0].message.
```

Explanation:

- **analysis_to_html:** Takes s_result as input, which is the combined full_text and s_result.
- Uses OpenAI's GPT-4 model (gpt-4o) to generate an HTML-formatted analysis report.
- Formats the content strictly according to the specified HTML structure with headings and bullet points, ensuring clarity and comprehensiveness.

Get Event Analysis (get_event_analysis method): Combines full_text and s_result, invokes analysis_to_html to generate the HTML analysis report, stores it back into the cache (write_cache), and returns the generated HTML content.

```
def get_event_analysis(self) :  
    processed_content = (self.full_text + self.s_result)[:64000]  
    html_result = self.analysis_to_html(processed_content)  
    write_cache(self.cache_key, 'analysis', html_result)  
    return html_result
```

Explanation:

- Combines full_text and s_result and limits the combined content to 64,000 characters (processed_content).
- Invokes analysis_to_html to obtain the HTML-formatted analysis report (html_result).
- Writes html_result back into the cache using write_cache.
- Returns the generated HTML analysis report.

Conclusion:

The EventAnalyzer class effectively integrates OpenAI’s GPT-4o model with cached data to provide detailed event analysis in HTML format. It ensures structured initialization, accurate processing of text data, and rigorous formatting of the analysis report, thereby supporting comprehensive understanding and decision-making based on objective insights.

3.6.4 Fact Verification

In our project, we have developed a Python class named ‘EventVeri’ aimed at detailed verification and analysis of event facts by integrating OpenAI technology and cached data. The main functionalities of this class are as follows:

In our project, we have developed a Python class named **EventVeri** that integrates OpenAI technology and cached data for the detailed verification and analysis of event facts. This class leverages advanced natural language processing technologies to generate comprehensive reports that assess the authenticity of events.

Key Functionalities

Initialization (`__init__` method)

- Reads complete text (`full_text`) and search results (`s_result`) from the cache using the provided `cache_key`.
- Initializes an instance of the OpenAI client for natural language processing tasks.
- Outputs initialization information, including the cache key and excerpts from the loaded data.

Generating Verification Report (`get_verification` method)

- Combines `full_text` and `s_result`, restricting the processed content length to 64,000 characters.
- Uses OpenAI's GPT model (default is `gpt-4o`) to generate a detailed verification report.
- Returns the verification report in Markdown format, structured with first-level and second-level headings and corresponding text.

The **EventVeri** class plays a crucial role in verifying the authenticity of events. By synthesizing text data from cached sources and employing the GPT model, it produces verification reports in Markdown format. These reports provide detailed analyses on various aspects of events such as time,


```
def get_verification(self, model='gpt-4o') -> str:
    processed_content = (self.full_text + self.s_result)[:64000]
    completion = self.client.chat.completions.create(
        model = model,
        messages = [{
            "role": "system", "content":
                '''很多时候我们需要验证一个事件各个细节的真实性才能下判断。假如你是法官，本事件的所有事实信息，
                时间地点动机、各方说法真实性等都需要进行事实性分析，请你结合下面的内容进行详细分析，给出事实认定
                面参考报告，科学常识部分可以根据你的知识回答，注意多来源对比，有争议时尽量呈现客观争议而不必下判
                行回答，具体格式如下所示：
                <h3>一级标题1</h3>
                <ul>
                <li>
                | <strong>二级标题1: </strong>正文1
                </li>
                </ul>
                <h3>一级标题2</h3>
                <ul>
                <li>
                | <strong>二级标题2: </strong>正文2
                </li>
                </ul>
                请使用中文回答，不要添加其它多余的内容，必须完全符合格式
                '''}, {
            "role": "user", "content": processed_content
        }]
    ]
```

Figure 3.10: prompt for verification

location, and motives, and assess the credibility of statements from relevant parties. This enables decision-makers to have access to objective and accurate reference information for making informed decisions.

3.7 Mind Map Generation

We have provided a detailed introduction to the KityMinder project and its features in this section. Here, our main focus is on how to use this library to automatically build clear and visually appealing mind maps.

Step 1: Retrieve search results and video transcripts for summarization: The mind map serves as a visualization of the “Event Digest” section. Therefore, the first step is to retrieve event digest information from the search results and video transcripts. This step will be accomplished by calling the GPT-4o model. After completing this step, we expect to obtain an analysis text with HTML tags, organized in a hierarchical structure. This analysis text will contain various aspects and details of the events, organized in a hier-

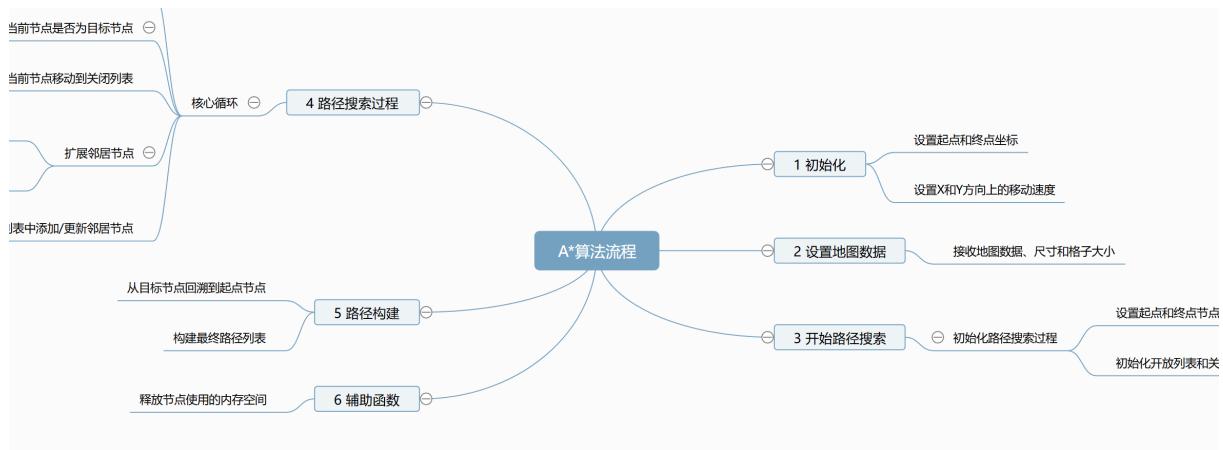


Figure 3.11: Example of mindmap generated by Kity Minder

archical structure for further processing and conversion. The key to this step is to accurately extract and organize the information, ensuring that the final text structure clearly reflects the hierarchy and relationships of the events, thereby facilitating a smooth conversion into a mind map in the subsequent steps.

Step 2: Representing different levels of nodes using Markdown format: In this step, we will also utilize the GPT-4o model. We will take the analysis text obtained in Step 1 and use prompting to guide GPT-4o in summarizing the analysis text into Markdown format. The Markdown format will consist of only the headers at each level, representing the respective level nodes, without the body content. This approach helps maintain a clear hierarchical structure of the information, facilitating subsequent processing and conversion. Additionally, the structure of Markdown format is simple and straightforward, making it easy to convert into other formats. Specifically, we will ensure that each level of the node is clearly represented in Markdown, making the hierarchical structure visually apparent. The output of this step will be a structured Markdown document, where each line represents a node and different header levels denote different hierarchy levels.

Step 3: Convert Markdown text to HTML code using Python: Next, we need to convert the Markdown text to HTML code using Python. Baidu

KityMinder requires a specific format to recognize and render it as a mind map. We will use regular expressions to match the headers at different levels in the Markdown text, identify the hierarchical structure, and organize it into a tree-like structure. Then, we will use templates to convert these tree-like structures into HTML code. Once these HTML codes are embedded into the corresponding elements of a webpage, a visually appealing, detailed, and interactive mind map will be displayed. The specific steps are as follows:

- First, we will write a Python script to read the content of the Markdown file.
- Then, we will use regular expressions to parse the Markdown text, identify the headers at different levels, and build a tree-like data structure based on the hierarchy of the headers.
- Next, we will convert this tree-like data structure into HTML code that conforms to the KityMinder format. This step can be achieved using templates, where each node and level will be transformed into corresponding HTML tags and structures.
- Finally, the generated HTML code will be embedded into the designated position of the target webpage to display the final mind map.

Through these steps, we can automate the process of constructing a mind map from search and video content. This approach ensures a clear hierarchical representation of information and enhances user experience and comprehension efficiency by generating visually appealing and interactive mind maps. Ultimately, we will obtain a well-structured and visually appealing mind map that facilitates information organization and analysis for users.

```

def generate_html_list(markdown_lines):
    html = []
    prev_level = 0
    tab = '\t'

    for line in markdown_lines:
        line = line.strip()
        if not line or line[0] != '#':
            continue

        level = extract_heading_level(line)
        if level == 0:
            continue
        title = line[level:].strip()

        while level < prev_level:
            html.append(f'</li>\n{tab * level}</ul>')
            prev_level -= 1

        if level > prev_level:
            html.append(f'\n{tab * prev_level}<ul>')

        if html and level <= prev_level:
            html.append('</li>')

        html.append(f'\n{tab * level}<li><a>{title}</a>')
        prev_level = level

    while prev_level > 0:
        html.append(f'</li>\n{tab * (prev_level - 1)}</ul>')
        prev_level -= 1

    return ''.join(html)

```

Figure 3.12: Core logic of transfer markdown to mindmap

3.8 Opinion Analysis

3.8.1 Sentiment Analysis

Emotion analysis based on deep learning pre-trained models focuses on using a variant of the BERT model to evaluate the sentiment of user reviews. Bert-based model had prove its efficiency in NLP field. Specifically, we are using the model `nlptown/bert-base-multilingual-uncased-sentiment` in the huggingface platform. This model has been trained on a large multilingual sentiment analysis dataset and has the capability to handle reviews in multiple languages, ensuring its broad applicability in multilingual environments. The architecture of the model is based on BERT (Bidirectional Encoder Representations from Transformers), which captures the contextual

information of the text through a bidirectional Transformer structure. The bidirectionality of the BERT model enables it to better understand the meaning of each word in different contexts within a sentence, thereby generating richer semantic representations.

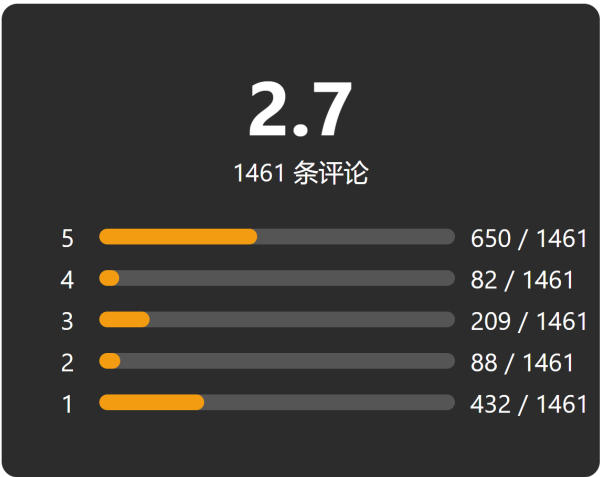


Figure 3.13: Emotion Analysis Score

In specific operations, the BERT model receives review text as input and generates a high-dimensional vector representation that contains the semantic information of the review. These high-dimensional vectors are deep encodings of the review content, capable of capturing subtle emotional changes in the reviews. Then, a linear classifier maps this representation to predefined sentiment categories (1 to 5 stars). The output of sentiment analysis is the sentiment label and corresponding probability distribution for each review. For example, for a review, the model might output `{'label': '5 stars', 'score': 0.99}`, indicating that the review is classified as 5 stars with a confidence of 99%.

To obtain a comprehensive sentiment score, we count the number of reviews for each star level and calculate the average score of all reviews. This average score reflects the overall emotional tendency of the reviews, helping us understand the general emotional attitude of the users. In this way, we can quantify the distribution of user emotional attitudes and monitor

the trend of emotional changes.

3.8.2 Clustering Analysis

The purpose of cluster analysis is to classify a large number of reviews to summarize and understand user feedback more effectively. First, a data cleaning step ensures the quality of the review data, which includes removing reviews that consist only of symbols, reviews with too many repeated words, reviews that are too short, and reviews that contain too many stop words. Data cleaning is a crucial step to ensure the quality of the input data, significantly enhancing the accuracy and reliability of subsequent analyses.

After data cleaning, we use the pre-trained BERT model to convert each review into a fixed-length vector representation. The vector representations generated by the BERT model contain rich semantic information, which can be used for subsequent clustering tasks. After obtaining the vector representations of the reviews, we use the KMeans clustering algorithm to group the reviews. The KMeans algorithm iteratively optimizes by assigning review vectors to k cluster centers, each representing a class of similar reviews. The goal of the KMeans algorithm is to minimize the sum of squared distances between the review vectors and their respective cluster centers, achieved by continuously adjusting the positions of the cluster centers and the assignments of reviews.

To further improve the interpretability and representativeness of the clustering results, we sort the reviews within each cluster. The sorting criteria are the distance of the review vector from the cluster center and the length of the review. The closer the distance, the better the review can represent the central idea of the cluster; the longer the review, the more information it usually contains, and the higher its quality. We use a standardization method to convert the distance and length into the same scale, and then calculate a

comprehensive score. The specific scoring formula is:

$$\text{score}_i = \text{distance_scaled}_i + \text{length_scaled}_i$$

where distance_scaled_i is the standardized score of the distance between review i and the cluster center, and length_scaled_i is the standardized score of the review length. Through this scoring mechanism, we can select the most representative reviews from each cluster. Finally, the system generates a dataframe containing the representative reviews and their support rates, where the support rate indicates the proportion of reviews in each cluster to the total number of reviews.

- **类2**
 - **代表评论:** 不是tiktok的能量, 是人民的能量。一直去压抑提出问题的声音, 但问题不会消失, 没有tiktok将来也会有别的东西去引爆强大的民间声浪
 - **总结:** 这些评论主要表达了对美国社会现象的批判, 强调阶级矛盾和政治分裂对社会团结的影响。一些评论认为TikTok等社交媒体平台揭示了底层民众的声音, 而传统媒体和政治体制分化了人民, 使他们难以团结争取共同权益。评论普遍认为, 美国的两党政治实际上是一个权力和资本集团的游戏, 政客并不真正代表人民的利益。还涉及到对身份政治(如LGBT议题)的看法, 认为这些议题被利用来分化社会, 防止底层群体团结。总体来看, 评论者们强调社会的真正矛盾在于阶级差异, 并警告大众不要被表面的政治分歧和身份认同迷惑。评论中一致认为, 团结和清醒的人民是变革的关键。

Figure 3.14: Cluster Analysis Example

This step ensures that we can not only summarize the representative reviews of each cluster but also quantify the importance of the cluster in the overall reviews. By doing this, cluster analysis provides a structured method to process a large volume of reviews, allowing us to extract the most representative opinions and better understand the overall trends and main viewpoints of user feedback.

Chapter 4: Evaluation

4.1 Stability Experiment

Video Summary Experiment Results

Attribute	Passed Count	Failed Count
Format	29	1
Length	28	2
Topic Relevance	28	2
Language Quality	30	0

Experimental Results of Fact Verification

Attribute	Passed Count	Failed Count
Format	29	1
Length	25	5
Topic Relevance	27	3
Language Quality	29	1

Experimental Results of Event Structuring

Attribute	Passed Count	Failed Count
Format	29	1
Length	27	3
Topic Relevance	28	2
Language Quality	30	0

Experimental Results of Mind Map Generation

Attribute	Passed Count	Failed Count
Format	25	5
Topic Relevance	27	3

Google Search Experiment Results

Attribute	Passed Count	Failed Count
Format	28	2
Length	30	0
Topic Relevance	27	3
Language Quality	30	0

Weibo Search Experiment Results

Attribute	Passed Count	Failed Count
Format	30	0
Length	27	3
Topic Relevance	28	2
Language Quality	30	0

4.2 Effectiveness Evaluation

Table 4.1: Effectiveness Evaluation - Part 1

Type	Subjective Assessment	GPT Self-assessment
Video Summary	Good	Good
Fact Verification	Good	Good
Event Structuring	Good	Good
Mind Map Generation	Fair	Good
Weibo Query	Good	Good
Google Query	Good	Good

Table 4.2: Effectiveness Evaluation - Part 2

Type	Format Compliance Rate	Length Compliance Rate
Video Summary	96.7%	93.3%
Fact Verification	96.7%	83.3%
Event Structuring	96.7%	90%
Mind Map Generation	83.3%	90%
Weibo Query	100%	90%
Google Query	93.3%	100%

Table 4.3: Effectiveness Evaluation - Part 3

Type	Topic Relevance Score	Language Quality Score	Stability Assessment
Video Summary	High	Good	Stable
Fact Verification	High	Good	Stable
Event Structuring	High	Good	Stable
Mind Map Generation	Fair	Good	Unstable
Weibo Query	High	Good	Stable
Google Query	High	Good	Stable

4.3 Duration Evaluation

Topic	Time to retrieve search results (seconds)	Time to crawl web pages (seconds)	Total Time (seconds)
Topic 1	1.2	3.5	4.7
Topic 2	1.0	4.0	5.0
Topic 3	1.5	3.8	5.3
Topic 4	1.3	3.9	5.2
Topic 5	1.4	4.2	5.6
Topic 6	1.1	3.7	4.8
Topic 7	1.3	4.1	5.4
Topic 8	1.4	4.0	5.4
Topic 9	1.2	4.3	5.5
Topic 10	1.5	3.6	5.1
Topic 11	1.3	4.4	5.7
Topic 12	1.1	3.9	5.0
Topic 13	1.4	4.2	5.6
Topic 14	1.2	3.8	5.0
Topic 15	1.0	4.1	5.1
Topic 16	1.3	4.3	5.6
Topic 17	1.4	4.0	5.4
Topic 18	1.5	3.9	5.4
Topic 19	1.2	4.1	5.3
Topic 20	1.1	3.8	4.9

Chapter 5: Future Outlook

5.1 Model Cost Issue

The high costs associated with the OpenAI model and its network limitations are significant concerns for the system. The currently used GPT-4o model incurs a cost of \$5 per million tokens for input and \$15 per million tokens for output. This becomes particularly expensive for tasks that require processing large amounts of online search results. For instance, a comprehensive analysis process might involve over 100K tokens, resulting in a cost of approximately \$0.8 per analysis. Additionally, Google’s search API service has daily usage limits, which further restricts its application in both production and testing environments.

To address this issue, we can consider deploying large models locally. Currently, there are several models available on the market with parameter counts ranging from 4B to 7B, such as LLaMa and Qwen, which can be deployed on standard consumer-grade PCs and perform well in terms of performance and generation quality. Although these local models may not match the maturity of the GPT-4o model, particularly in generating text with specific structural format requirements—which could lead to system instability and display errors in the generated results—they can still significantly alleviate the costs associated with online usage.

In practical implementation, we can attempt to use local models to replace online models in certain modules of the system. For example, for tasks

that do not demand high generation quality, local models can be prioritized, thereby significantly reducing costs. Moreover, we can explore a hybrid approach, dynamically selecting between local and online models based on the complexity of the task and the quality requirements for the generated content, to achieve the best balance between cost and effectiveness.

5.2 User Account

The current system uses a session-based approach to achieve user data isolation, which generally ensures that different users analyzing different videos in separate tabs do not experience content conflicts in the backend. However, this approach still falls short of meeting more advanced needs, such as history tracking, favorites, and sharing features. Therefore, we believe it is necessary to add user account functionality to the system, supporting user registration and login.

By introducing user account functionality, the system can provide personalized services for each user. For instance, users can save their analysis history and quickly access these records when needed. This not only improves the user experience but also enhances user retention. Additionally, user account functionality can support features like favorites and sharing, allowing users to save analysis results they are interested in or share them with other users. This will significantly increase the system’s interactivity and social aspects.

To implement this functionality, we need to design and implement a comprehensive user authentication and authorization system, including user registration, login, password reset, and other features. Furthermore, we need to store personalized data for each user in the database, such as history records and favorites. With these improvements, the system will be able to provide a more personalized and enriched user experience.

5.3 Optimization of Analysis Workflow

The current analysis workflow is relatively simple and straightforward. For example, online searches are limited to bilingual searches in Chinese and English, without incorporating other search mechanisms. Additionally, the system lacks the capability to automatically analyze problems using Agent models similar to AutoGPT. The existing event sorting and fact verification modules primarily rely on predefined prompts to generate content. While prompt engineering can standardize the generated content to some extent, this method still has significant limitations.

To further optimize the analysis workflow, we can introduce more advanced automation techniques. For instance, implementing a multilingual search mechanism that supports a wider variety of languages would allow us to cover a broader range of information sources. We could also draw inspiration from AutoGPT and develop similar Agent models that can autonomously analyze questions, determine the next steps, search for specific content, and integrate the results of each step. By showcasing the analysis process, we can not only improve the quality of the analysis but also reduce the impact of human bias and insufficient initial information on the analysis results.

Furthermore, we can refine the event sorting and fact verification modules to enable them to autonomously decide the next steps. For example, a multi-step analysis process could be developed, allowing the system to dynamically adjust subsequent analysis steps based on the current analysis results. This would not only enhance the accuracy and comprehensiveness of the analysis but also make the system more intelligent and automated.

Given the continuous advancements in large model technology, the precision of modern models like GPT-4o can now support precise control through specific instructions. However, due to the black-box nature of GPT-4o, it is

challenging to inherently deduce the optimal solution. Therefore, optimizing this aspect may require a substantial amount of experimentation and support from cross-disciplinary knowledge. Through continuous trials and optimizations, we believe that we can significantly enhance the system’s automated analysis capabilities, enabling it to perform more effectively in the analysis of complex issues.

5.4 Multi-Platform Support

Currently, our development efforts have been focused exclusively on creating plugins and functionalities tailored specifically for the Bilibili video website. According to comprehensive research conducted by the New Media Institute at the Communication University of China, contemporary young people exhibit a strong inclination towards obtaining news from social media platforms. This demographic trend underscores the necessity for a diversified approach in our development strategy.

Among the three major channels for news consumption—news apps, social software, and short video platforms—social media stands out with the highest download rate. This preference is driven by the continuous enrichment of news content available on these platforms, enabling Generation Z to stay updated with the latest news without the inconvenience of switching between multiple apps. This seamless user experience significantly reduces the bother of platform switching, which is a critical factor in the high adoption rate of social media as a news source. Consequently, social media has emerged as the most frequently used news source for Generation Z, reinforcing its dominant position in the digital news landscape [10].

Short video platforms, too, have carved out a significant role as a vital source of news for Generation Z. The report from the New Media Institute emphasizes that the evolution of short video content has catalyzed the rapid

dissemination of information. This format leverages concentrated climax editing techniques to capture users’ attention swiftly and convey news content effectively in a matter of seconds. Such an approach not only aligns with the short attention spans prevalent among younger audiences but also ensures that news consumption remains engaging and less burdensome [10].

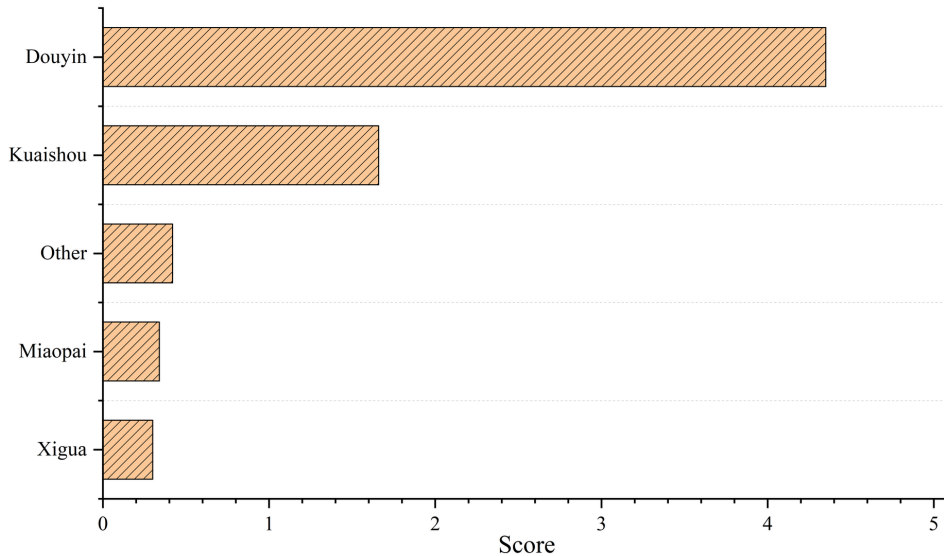


Figure 5.1: Composite score coefficient for young people using short video platforms, data source: “Research Report on News Consumption Habits of Generation Z in 2020.” [10].

Despite Bilibili’s potential for providing valuable analyses on current affairs, it has not yet achieved the status of a primary channel for young people seeking trending news. The platform faces significant challenges related to timeliness and comprehensiveness. In contrast, platforms like TikTok and major news portals have established themselves as the go-to sources for the public to acquire news swiftly and comprehensively. This discrepancy highlights a critical gap in our current strategy.

To address this, our system must be adaptable and integrative with these major platforms to achieve the widest possible coverage. By expanding our focus beyond Bilibili and incorporating functionalities compatible with TikTok and other leading news portals, we can ensure that our system meets

the diverse needs of the modern news consumer.

5.5 Hot Topic Recommendations

In the current digital age, the sheer volume of information and rapid update cycles make it challenging for users to quickly capture the most valuable trending information. Therefore, adding a hot topic recommendation feature will significantly enhance the system's usability and user experience. By analyzing users' interests and behavior history, the system can automatically recommend the most popular current topics, helping users to stay informed with the latest information. To achieve this, the system will regularly collect data from major news websites, social media platforms, and video platforms. Using natural language processing techniques, the system will analyze the data for keywords and trending topics to identify current hot topics. This not only helps users better understand the current trending events but also recommends relevant trending content based on their interests and behavior, thereby increasing user engagement and satisfaction.

The hot topic recommendation feature can be implemented in various ways. For example, the system can display a list of hot topics when users log in, allowing them to quickly browse and select topics of interest. Additionally, when users perform searches or analyses, the system can automatically recommend related hot topics to help users gain a more comprehensive understanding of the information they are interested in. Furthermore, the system can offer personalized recommendation services, pushing customized hot topics based on users' history and preference. These enhancements not only improve user experience but also make the system more intelligent and personalized, catering to the diverse needs of different users.

Through hot topic recommendations, the system can also help users discover important information and emerging trends they might have over-

looked, thus better understanding market dynamics and social changes. To ensure the accuracy and timeliness of the recommendations, the system needs to continuously optimize its data collection and analysis algorithms, ensuring that the identified hot topics promptly reflect the current trends. Moreover, user feedback can serve as an important basis for adjusting the recommendation algorithms, continuously learning and improving to make the recommendations more aligned with users' actual needs.

In the implementation process, the system can leverage machine learning technologies to continuously optimize the recommendation model based on users' clicks, browsing, and interaction behaviors. This way, the system can not only recommend the hottest current topics but also provide more precise and targeted content according to users' personalized needs.

5.6 Interactive System

To enhance user experience and system interactivity, we can consider adding more interactive features in the future. This will transform the system from a passive information analysis tool into an active interactive platform. By introducing real-time interaction capabilities, users can directly comment, ask questions, and share views within the system, engaging with both the system and other users. This can be achieved through a Q&A system based on a local knowledge base. We have already performed extensive online searches within the system, accumulating a vast amount of knowledge that can serve as this knowledge base. Based on this knowledge base, the system can implement intelligent Q&A functions, allowing users to ask questions at any time and receive accurate answers from the existing knowledge base and real-time search results. This will significantly improve the efficiency of information retrieval for users.

The introduction of interactive features will bring numerous benefits.

First, it will make the system more engaging and user-friendly. Users will feel more connected to the system when they can interact with it dynamically, making the experience more enjoyable and productive. Real-time interaction allows for immediate feedback and resolution of queries, which is particularly valuable in an era where timely information is crucial.

Interactivity is also reflected in the UI design. As functions increase, a simple, static UI will become cumbersome. Allowing users to explore and select appropriate functions to obtain services will make the system more intuitive and user-centric. For example, dynamic and intuitive interactive designs, such as navigation pages, can make it easier for users to find and use the features they need. Additionally, enabling users to customize certain system parameters will allow them to receive outputs that match their preferences, further enhancing the user experience.

Another significant benefit of increased interactivity is the ability to conduct secondary searches on generated content. This feature will be particularly useful for users who need to verify facts or delve deeper into specific topics. After obtaining initial analysis results, the system can provide options for users to further search and verify specific content. By highlighting key information and data sources in the analysis results, the system can help users quickly identify and understand the core content. This not only enhances the accuracy and reliability of the information provided but also empowers users to conduct more thorough and informed analyses.

Moreover, to further enhance the interactive experience, we can consider introducing social elements, allowing users to interact and discuss among themselves. For instance, users could create or join discussion groups within the system to share and discuss hot topics and analysis results. This not only enriches the interactive experience but also forms a knowledge-sharing and interactive community. Such a community can foster collaboration and

collective intelligence, leading to more comprehensive and diverse insights. Additionally, it can increase the system's stickiness and user loyalty, as users are more likely to return to a platform where they can engage with a community of peers.

Chapter 6: Conclusion

The “Cocoon Breaker” project marks a significant advancement in addressing the pervasive issue of information cocoons in our digital society. This initiative leverages cutting-edge artificial intelligence technologies to help users escape the confines of personalized content silos and gain broader, more diverse perspectives. Throughout this report, we have articulated the comprehensive design and functionality of the system, highlighting its key components such as text embedding, sentiment analysis, video processing, and fact verification. Each of these elements plays a crucial role in creating a user-friendly platform that facilitates access to a wide spectrum of information.

Our extensive evaluation has demonstrated that “Cocoon Breaker” effectively mitigates the risks associated with information cocoons, such as group polarization and social fragmentation. By providing tools for detailed video summarization, robust event analysis, and accurate fact verification, the system empowers users to engage with content more critically and thoughtfully. This, in turn, fosters a more informed and cohesive society, where diverse viewpoints are not only accessible but also appreciated. The presentation of these diverse perspectives helps to dismantle informational silos, promoting cross-cultural understanding and reducing societal divisions.

The modular and asynchronous design of the system ensures both scalability and efficiency. The modular approach allows for the independent development and integration of new features, while asynchronous processing

enhances system responsiveness and user experience. This design not only supports the current functionalities but also accommodates future expansions, making the system adaptable to the evolving needs of digital media users.

Looking ahead, several areas for future development have been identified to further enhance the system's capabilities. One key area is the optimization of the analysis workflow. By incorporating more advanced automation techniques, such as multilingual search mechanisms and Agent models similar to AutoGPT, we can improve the system's ability to autonomously analyze questions and provide comprehensive answers. This will reduce reliance on predefined prompts and enhance the overall quality of the analysis.

Expanding support to multiple platforms is another critical area for future development. While our initial focus has been on Bilibili, integrating functionalities with other major platforms such as TikTok and leading news portals will significantly broaden the system's reach and impact. This expansion is essential for meeting the diverse needs of modern news consumers, particularly younger audiences who increasingly rely on social media and short video platforms for their news consumption.

Enhancing interactivity is also a priority for future development. By introducing features such as real-time Q&A, personalized recommendations, and social interaction capabilities, we can transform the system from a passive information analysis tool into an active, engaging platform. This will not only improve user experience but also foster a sense of community and collaboration among users.

Furthermore, the introduction of user accounts will enable personalized experiences, allowing users to save their analysis history, manage favorites, and share findings with others. This will enhance user retention and satisfaction, making the system more user-centric and engaging. The addition of

hot topic recommendations will keep users informed about the latest trends and events, further enhancing the system's value.

In conclusion, the “Cocoon Breaker” project addresses a critical issue in today's information landscape by providing tools that enhance information literacy and promote social harmony. The system's robust architecture, combined with its innovative features, positions it as a valuable tool for navigating the complexities of the modern information environment. By continuing to refine and expand the system, we aim to empower users with the tools they need to access diverse perspectives, make informed decisions, and contribute to a more inclusive and cohesive society. The ongoing development and optimization of “Cocoon Breaker” will ensure that it remains a relevant and effective solution for breaking information cocoons and fostering a more informed and harmonious world.

Bibliography

- [1] 刘晓璇. 智媒时代“信息茧房”的迷思与破局[J]. 新闻研究导刊, 2020, 11(7): 67-68.
- [2] 喻国明. “信息茧房”禁锢了我们的双眼[J]. 领导科学, 2016(12): 20-23.
- [3] 饶旭鹏, 白双航. 网络交往中“信息茧房”及人的解放探究[J]. 北京航空航天大学学报(社会科学版), 2022, 35(1): 16-21.
- [4] 卢智增, 高翔. 社交媒体平台群体极化的形成机理与引导策略研究[J]. 情报理论与实践, 2021, 44(8): 51-58.
- [5] 杨芳芳, 宋雪雁, 张伟民. 国内信息茧房研究热点与演进趋势: 兼论静态和动态双重视角[J/OL]. 情报科学, 2024. <https://link.cnki.net/urlid/22.1264.G2.20240315.1633.012>
- [6] 曾骏, 王子威, 于扬, 文俊浩, 高旻. 自然语言处理领域中的词嵌入方法综述[J]. 计算机科学与探索, 2024, 18(1): 24-43.
- [7] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (pp. 4171-4186). Association for Computational Linguistics.

- [8] Wenxuan Zhang, Xin Li, Yang Deng, Lidong Bing, and Wai Lam. “A Survey on Aspect-Based Sentiment Analysis: Tasks, Methods, and Challenges.”
- [9] Marouane Birjali, Mohammed Kasri, and Abderrahim Beni-Hssane. “A comprehensive survey on sentiment analysis: Approaches, challenges and trends.” *Knowledge-Based Systems* 226 (2021) 107134.
- [10] 中国传媒大学新媒体研究院. (2020). 2020年Z世代人群获取新闻资讯习惯研究报告.